Docket UI User Guide

The explorer and the workbench: every screen, every workflow, with screenshots, numbered steps and highlighted regions. For assessors, developers and users of the Uinsure Policies build.

Short form: cut to a short form on 2026-10-23 for time. The web app is not on this guide's branch yet (F8). Every `src/Uinsure.Policies.Web` path resolves on `feature/ui/U10`, and on `main` once the ui plan merges.

The big ideas

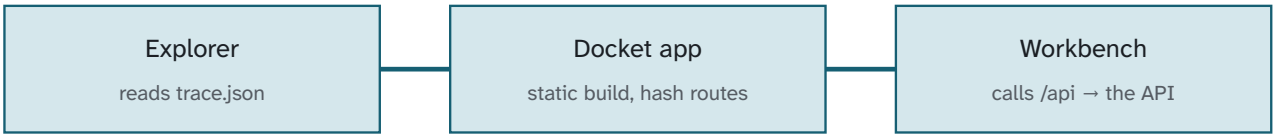
1. **Two halves.** The explorer reads the build trail (`trace.json`); the workbench drives the API with real requests.
2. **Evidence.** Every rule, node, commit, verdict and finding has a page and a deep link, from the requirement to its tests and commits.
3. **Honest.** A missing value is drawn as missing, an API answer is shown with the status the page received, and nothing is invented.
4. **Regions.** Each section of a screen carries a `data-region` tag; the numbered markers in this guide are measured from those tags.
5. **Contracts.** Screens were agreed as static HTML first and the app is tested against them: unit, end to end, axe, parity and visual regression.
6. **Run it yourself.** Every test collection can be run by hand: Postman, Playwright, Vitest.

1. What Docket is

Docket is one app with two halves: an explorer for the build trail and a workbench for the API.

The explorer reads `trace.json`, compiled from git and the trail files, and shows rules, plans, nodes, commits, verdicts and findings. The workbench sends sell, quote, cancel, claim and renew requests to the running API and shows the answers. Both are static pages with hash routes.

▼ In depth



Docket's two halves

Half	For whom	Reads	Needs
Explorer (18 screens)	Assessors and developers checking how it was built	`trace.json`	Nothing running
Workbench (7 screens)	Users and testers trying the API	The live API	The API on its port

Check your understanding

Question	Answer
Which half needs the API?	Only the workbench.
Where does the explorer's data come from?	`trace.json`, compiled by `npm run trace`.

2. Run it

Build once, preview it, and point `/api` at the API.

The static build is served by `vite preview`; the workbench calls `/api/...` and the app proxies it to the API port in `DOCKET_API_PORT` (default 5080). The dev server does the same on port 4170, which is what the Visual Studio launch profile starts beside the API.

▼ In depth

1. Install and build the app.

```
npm --prefix src/Uinsure.Policies.Web ci
npm --prefix src/Uinsure.Policies.Web run build
```

You should see: A `dist` folder.

```
(not captured in this short form)
```

2. Start the API (its http profile is port 5080), then preview the app.

```
dotnet run --project src/Uinsure.Policies.Api --launch-profile http
DOCKET_API_PORT=5080 npm --prefix src/Uinsure.Policies.Web run preview -- --
port 4170
```

You should see: The preview serves the app.

```
(not captured: the guide's screenshots came from a copy already running with
the app on port 4186 and the API on 5096)
```

3. Check both answer.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:4186/
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:5096/health/live
```

You should see: HTTP 200 from each, captured from that running copy.

```
200
200
```

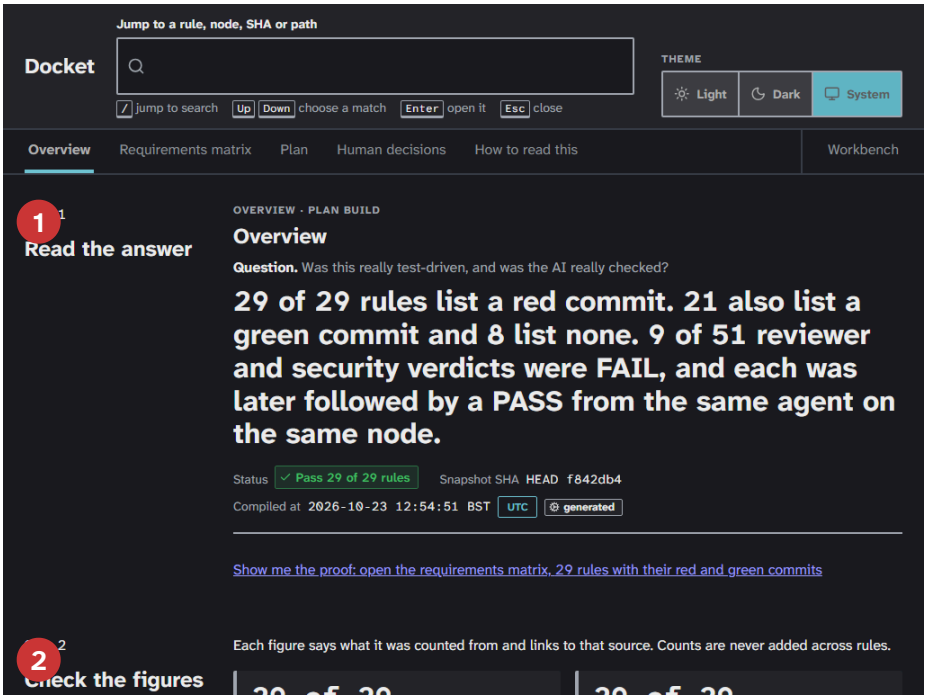
4. Open `http://localhost:4170/#/`.

You should see: The Overview. The workbench is at `#/workbench`.

5. Or use Visual Studio: open `Uinsure.Policies.slnx` and start the "API and web" profile.

You should see: Both start; the dev server's `/api` proxy defaults to port 5080.

Deep links are plain hashes, so any screen can be bookmarked or shared: `#/rules/CANCEL-1b`, `#/commits/<sha>`, `#/workbench/policies/<reference>`. The theme follows the system unless the header's Light or Dark button is pressed.



The same Overview in the dark theme (dark theme, 1024 px wide, `#/`)

#	Region (its data-region tag)	Proves
1	Headline answer	NAV-1
2	Receipt figures rules built total tests passing nodes done attempts and fails human deci…	NAV-1 VERIFY-1

Below this view: Build plan at a glance; Latest human decisions; Compiled at stamp; Top bar search and theme toggle global tagged here once; Trail completeness trail incomplete reasons verbatim not built yet and f0a as git only.

src/Uinsure.Policies.Web/vite.config.ts:21

```
const proxy = {
  '/api': {
    target: `http://localhost:${process.env.DOCKET_API_PORT ?? 5080}`,
    rewrite: (path: string) => path.replace(/^\/api/, ''),
  },
};
```

src/Uinsure.Policies.Web/Docket.Web.esproj:4

```
    <StartupCommand>npx vite --port 4170 --strictPort</StartupCommand>
```

> **Static Web Apps**
> The hosting config is infrastructure/web/staticwebapp.config.json:1 and the pipeline's web stage deploys it. It is evidence only: it was validated locally and never deployed.

Check your understanding

Question	Answer
Which port does the workbench's API call go to?	`DOCKET_API_PORT`, default 5080.
Is the Static Web App live?	No. Evidence only, never deployed.

3. Explorer tour

Eighteen screens that answer one question each about how the API was built.

Each screen below has a screenshot with numbered markers, a key of the regions the app tags, the rules it proves, numbered steps and the code that draws it. Regions starting below the first screenful are listed under the figure.

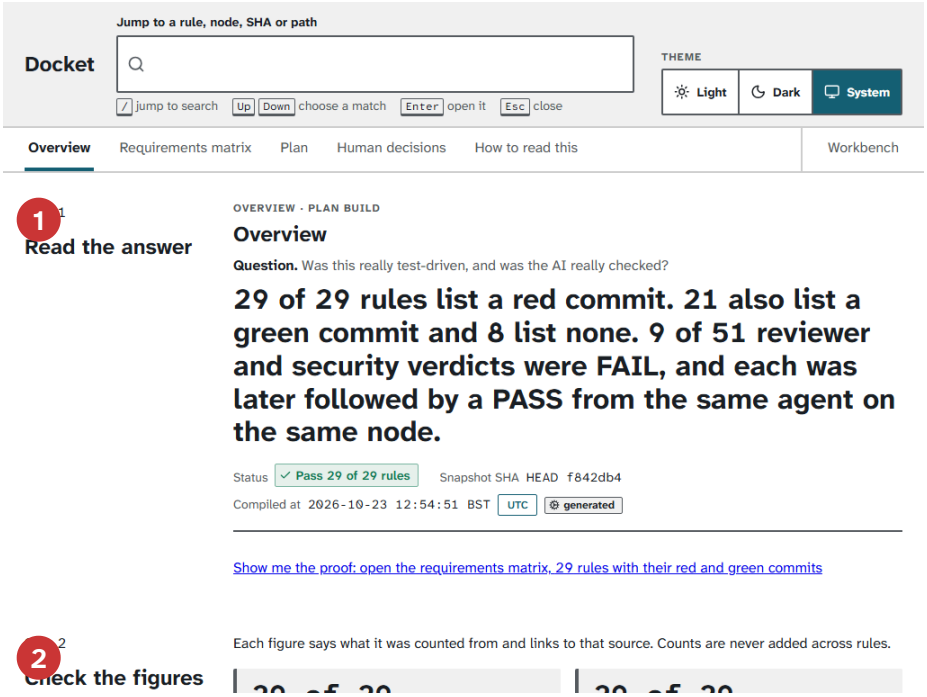
▼ In depth

> **Why**
> Each region carries `data-region` and `data-req`. Choice: measure markers from the tags. Alternative: place them by hand. Reason: a hand-placed marker drifts when the layout changes; a measured one cannot lie about where a tagged region is.

Overview

Was this really test-driven, and was the AI really checked?

The front door: one answer in a sentence, with a link to the proof. `#/` Rules and requirements it proves:
NAV-1 VERIFY-1 DAG-1 HITL-1 DATA-3 NAV-4 THEME-1 DATA-4



Overview (light theme, 1024 px wide, `#/`)

#	Region (its data-region tag)	Proves
1	Headline answer	NAV-1
2	Receipt figures rules built total tests passing nodes done attempts and fails human deci...	NAV-1 VERIFY-1

Below this view: Build plan at a glance; Latest human decisions; Compiled at stamp; Top bar search and theme toggle global tagged here once; Trail completeness trail incomplete reasons verbatim not built yet and f0a as git only.

1. Open `#/`.

   You should see: The answer sentence (marker 1) and its status.

2. Read the figures under the answer.

   You should see: Each figure names what it was counted from and links to it.

3. Press the proof link.

   You should see: The requirements matrix opens.

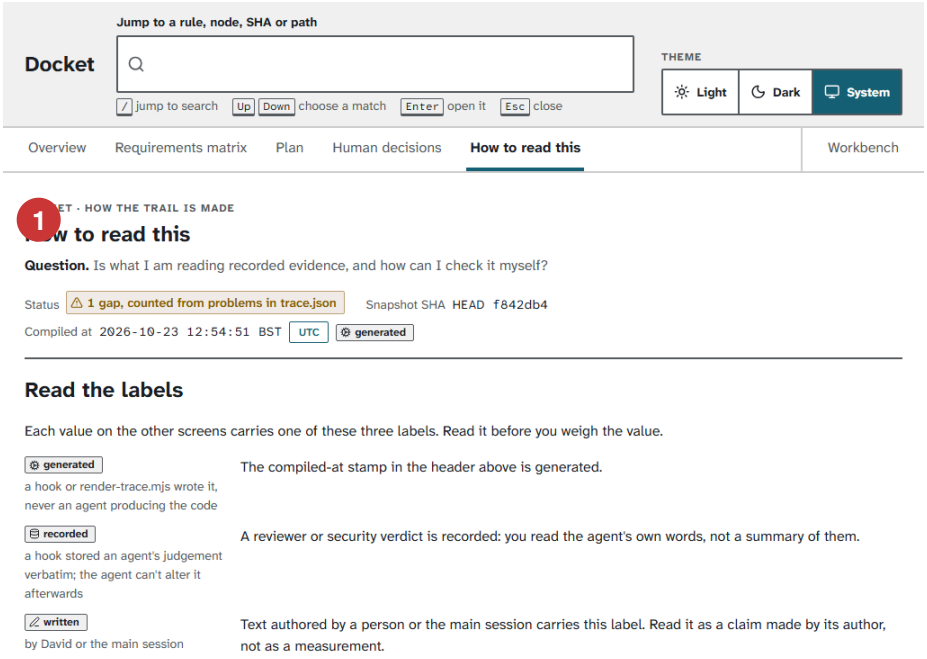
`src/Uinsure.Policies.Web/app/explorer/overview/Overview.tsx:134`

```
export function Overview() {
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
  const { trace, snapshot } = loaded;
```

How to read this

How do I read these pages?

The glossary and the provenance labels (generated, live, not returned). `#/about` Rules and requirements it proves: DATA-2 A11Y-1 VERIFY-1 NAV-5 DATA-3



How to read this (light theme, 1024 px wide, `#/about`)

#	Region (its data-region tag)	Proves
1	Provenance labels	DATA-2

Below this view: Glossary and status legend; Re verify with git; Guided tour entry one rule end to end could; Compiled at stamp the commit sha and utc time trace json was compiled from.

1. Open `#/about` from the nav.

   You should see: The provenance labels (marker 1).

2. Scan the glossary and status legend.

   You should see: Each status word with its colour and meaning.

3. Return to the Overview.

   You should see: The labels you learnt appear on every page.

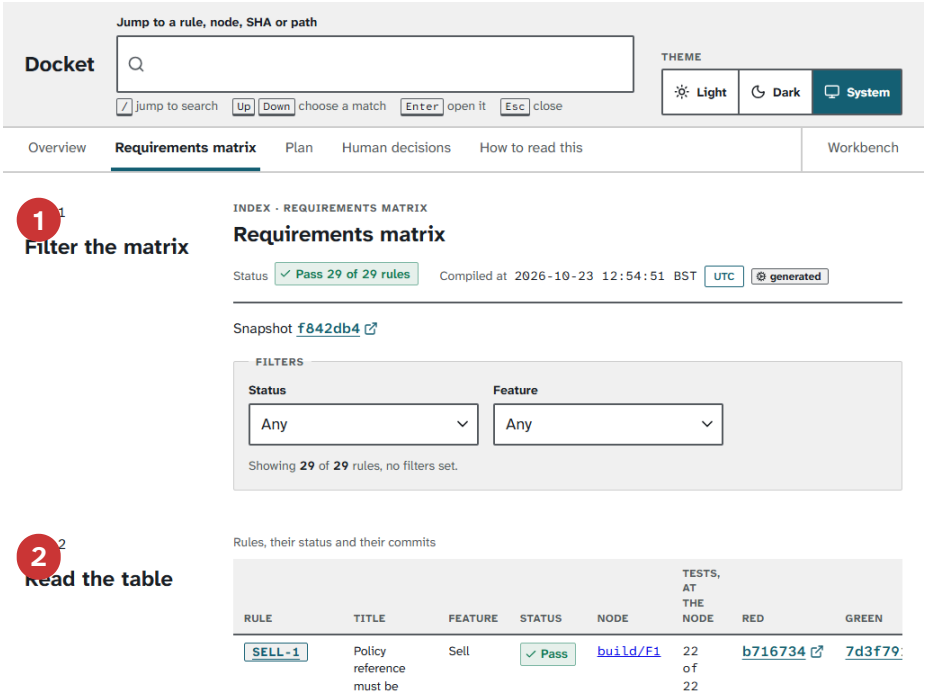
src/Uinsure.Policies.Web/app/explorer/about/About.tsx:156

```
export function About() {
  const { trace } = useShell();
  const gaps = trace.state === 'ready' ? gapsOf(trace.trace).length : null;
  return (
```

Requirements matrix

Which rules have red and green evidence?

Every rule with its red and green commits, filterable. `#/rules` Rules and requirements it proves: MATRIX-1 DATA-4



Requirements matrix (light theme, 1024 px wide, `#/rules`)

#	Region (its data-region tag)	Proves
1	Filters status feature	MATRIX-1
2	Matrix table	MATRIX-1

Below this view: Test count scope note counts are per node sell 7 10 at f1 vs 15 traited.

1. Open `#/rules`.

   You should see: Filters (marker 1) above the matrix table (marker 2).

2. Filter by status or feature.

   You should see: The table narrows.

3 Select a rule chip.

You should see: The rule page opens.

`src/Uinsure.Policies.Web/app/explorer/matrix/Matrix.tsx:152`

```
export function Matrix() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  return <MatrixTables trace={trace.trace} snapshotSha={trace.snapshot.sha}
snapshotShort={trace.snapshot.short} utc={trace.snapshot.utc} />;
```

One rule

What did this rule say, and what proves it?

A rule from its agreed text to its tests, commits and verdicts: the rule's evidence. `#/rules/CANCEL-1b` Rules and requirements it proves: RULE-1 RULE-2 HITL-1 RULE-3 VERIFY-1 BUSY-1 NAV-3



One rule (light theme, 1024 px wide, `#/rules/CANCEL-1b`)

#	Region (its data-region tag)	Proves
1	Rule text and agreed interpretation	RULE-1

Below this view: Signature red before green evidence custody chain tdd ticks chronology per style; Tests count and scope; Qa step and sign off; Red and green diffs lazy; Verify it yourself; Load diff busy while the diff chunk loads; Links out commit shas pr numbers and file line references to github at that sha.

1. Open `#/rules/CANCEL-1b`, or type the id in the search box.

   You should see: The rule text and agreed interpretation (marker 1).

2. Read the red-before-green evidence.

   You should see: The red commit comes before the green one (marker 2).

3. Follow a commit link.

   You should see: The commit page opens.

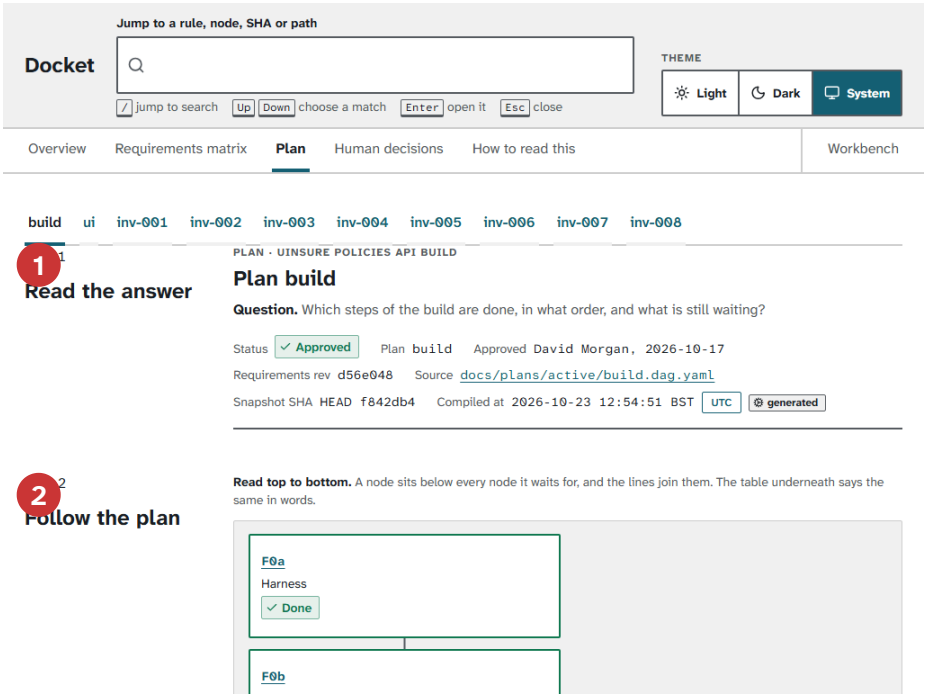
`src/Uinsure.Policies.Web/app/explorer/rule/Rule.tsx:309`

```
export function Rule() {
  const { ruleId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

A plan

What was built, in what order?

A plan's header, approval and node graph with its table. `#/plans/build` Rules and requirements it proves: **DAG-1** **LOOP-2**



A plan (light theme, 1024 px wide, `#/plans/build`)

#	Region (its data-region tag)	Proves
1	Plan header status approval requirements rev	**DAG-1**

#	Region (its data-region tag)	Proves
2	Node graph and its table	DAG-1

Below this view: Investigation body inv plans only.

1. Open `#/plans/build` through the Plan nav item.

   You should see: Plan header, status and approval (marker 1).

2. Read the node graph.

   You should see: Dependencies between nodes.

3. Select a node.

   You should see: The node page opens.

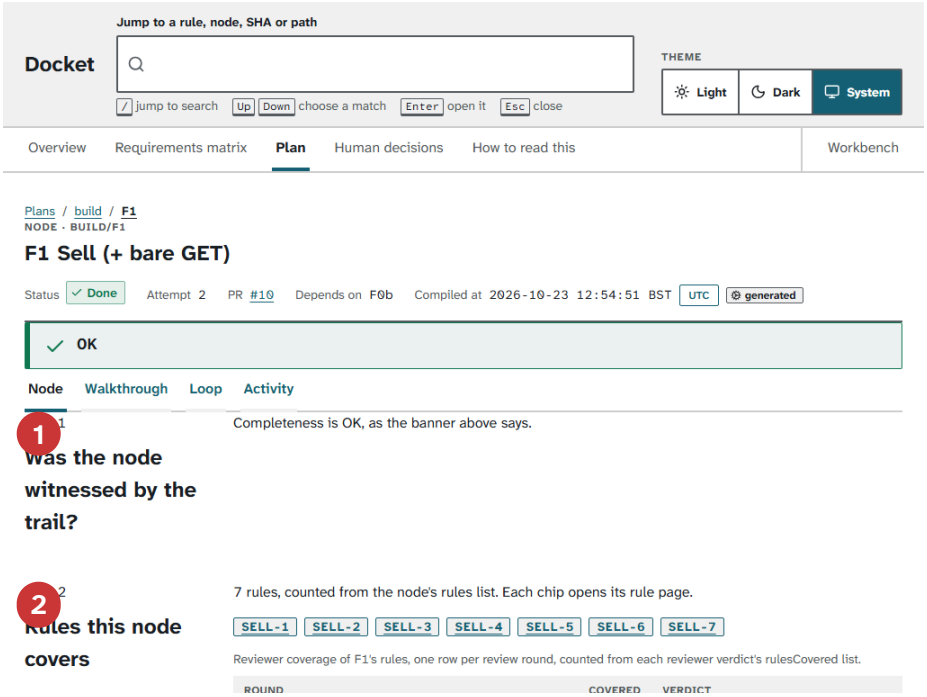
src/Uinsure.Policies.Web/app/explorer/plan/Plan.tsx:32

```
export function Plan() {
  const { planId = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
```

A node

Was the node witnessed by the trail, and what did it cover?

One node: completeness, the rules it covers and its tasks. `#/plans/build/F1` Rules and requirements it proves: DATA-4 DATA-8 NODE-1 LOOP-2 AGENT-1



A node (light theme, 1024 px wide, `#/plans/build/F1`)

#	Region (its data-region tag)	Proves
1	Completeness banner	DATA-4 DATA-8
2	Rules	NODE-1

Below this view: Tasks; Attempts; Who touched what.

1. Open the node, `#/plans/build/F1`.

   You should see: Completeness banner (marker 1).

2. Read the rules and tasks.

   You should see: Each rule is a chip that links to its page.

3. Use the tabs: Walkthrough, Loop, Activity.

   You should see: Each tab is its own screen below.

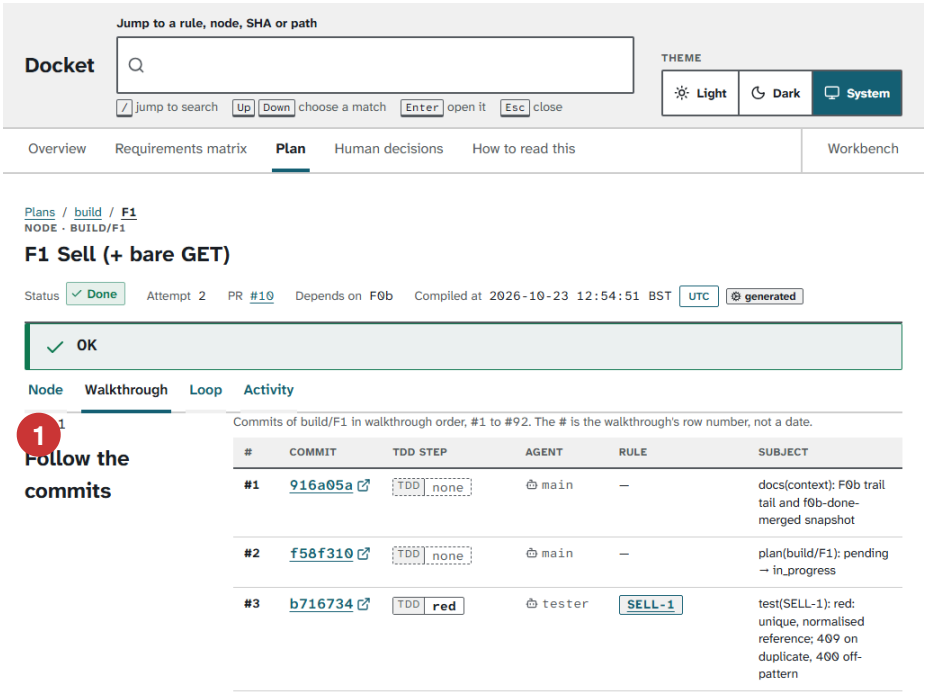
`src/Uinsure.Policies.Web/app/explorer/node/Node.tsx:27`

```
export function Node() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

A node's walkthrough

In what order did the commits land?

The node's commits in sequence, tagged red, stub, green or refactor. `#/plans/build/F1/commits` Rules and requirements it proves: **TDD-1**



A node's walkthrough (light theme, 1024 px wide, `#/plans/build/F1/commits`)

#	Region (its data-region tag)	Proves
1	Commit sequence	TDD-1

1. Open the Walkthrough tab of a node.

   You should see: The commit sequence (marker 1).

2. Read the TDD tag of each commit.

   You should see: Red precedes green.

3. Open a commit.

   You should see: The commit page opens.

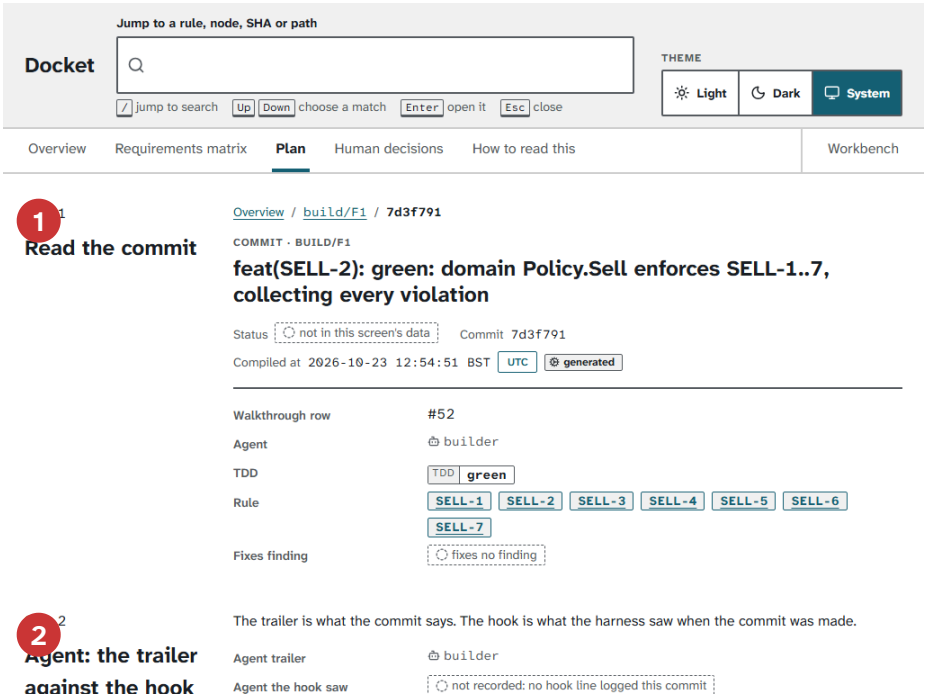
`src/Uinsure.Policies.Web/app/explorer/walkthrough/Walkthrough.tsx:43`

```
export function Walkthrough() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
```

One commit

What did this commit change, and who made it?

A commit's header and trailers, whether the trailer's agent matches the hook's, and its lazy diff.
`#/commits/7d3f791` Rules and requirements it proves: NAV-2 AGENT-1 RULE-3 NAV-3



One commit (light theme, 1024 px wide, `#/commits/7d3f7914e32e01f4a4a3d4a6d75dcfe60f94d668`)

#	Region (its data-region tag)	Proves
1	Commit header and trailers	NAV-2
2	Trailer agent vs hook agent	AGENT-1

Below this view: Diff lazy; Links out.

1. Open a commit from the walkthrough.

   You should see: Header and trailers (marker 1).

2. Compare trailer agent with hook agent.

   You should see: The two agree or the page says they do not.

3. Open the diff.

   You should see: The diff loads on demand.

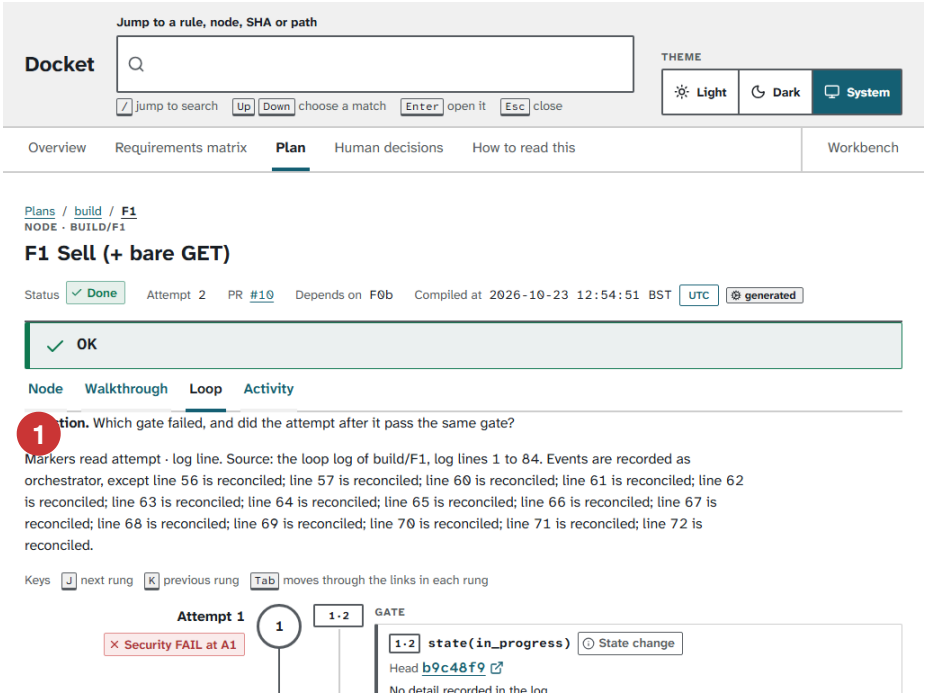
`src/Uinsure.Policies.Web/app/explorer/commit/Commit.tsx:83`

```
export function Commit() {
  const { sha = '' } = useParams();
  const { trace: loaded } = useShell();
  if (loaded.state !== 'ready') return <DataNotice trace={loaded} />;
```

A node's loop

How many attempts did the node take?

The attempt ladder: red, green, review, fix. `#/plans/build/F1/loop` Rules and requirements it proves: LOOP-2 LOOP-1



A node's loop (light theme, 1024 px wide, `#/plans/build/F1/loop`)

#	Region (its data-region tag)	Proves
1	Attempt ladder	LOOP-2

Below this view: Gate timeline.

1. Open the Loop tab.

   You should see: The attempt ladder (marker 1).

2. Read a rung.

   You should see: Each attempt and its outcome.

3. Follow a verdict link.

   You should see: The verdict page opens.

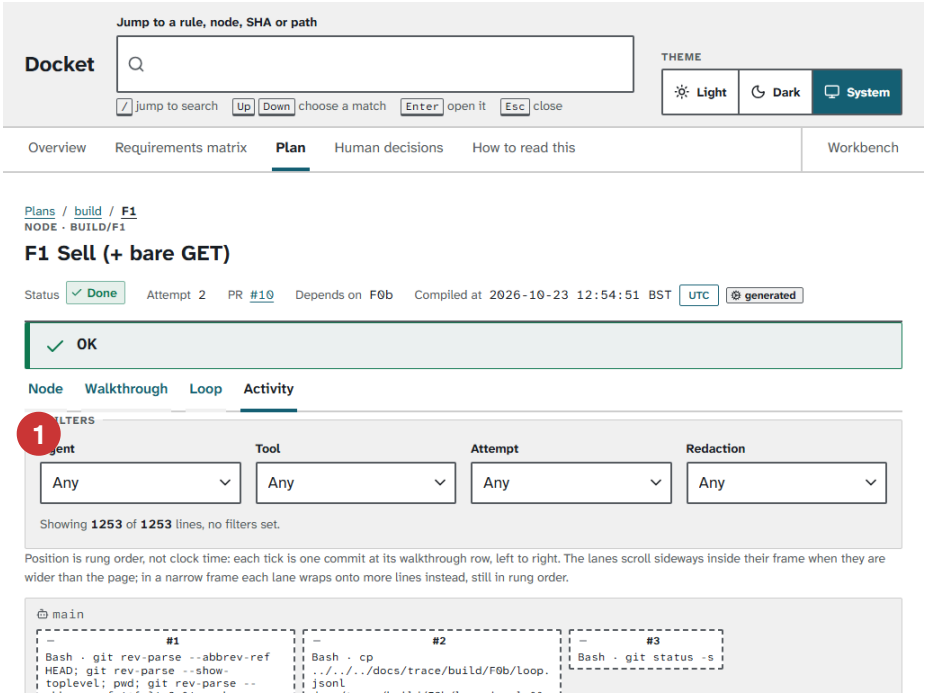
`src/Uinsure.Policies.Web/app/explorer/loop/Loop.tsx:63`

```
export function Loop() {
  const { planId = '', nodeId = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

A node's activity

Which agent did what, when?

Agent lanes over time, with a text equivalent table. `#/plans/build/F1/activity` Rules and requirements it proves: **AGENT-2** **A11Y-1**



A node's activity (light theme, 1024 px wide, `#/plans/build/F1/activity`)

#	Region (its data-region tag)	Proves
1	Agent lanes	**AGENT-2**

Below this view: Event table text equivalent.

1. Open the Activity tab.

   You should see: The agent lanes (marker 1).

2. Read the event table.

   You should see: The same events as text, for keyboard and screen-reader users.

3 Pick an event.

You should see: It links to its commit.

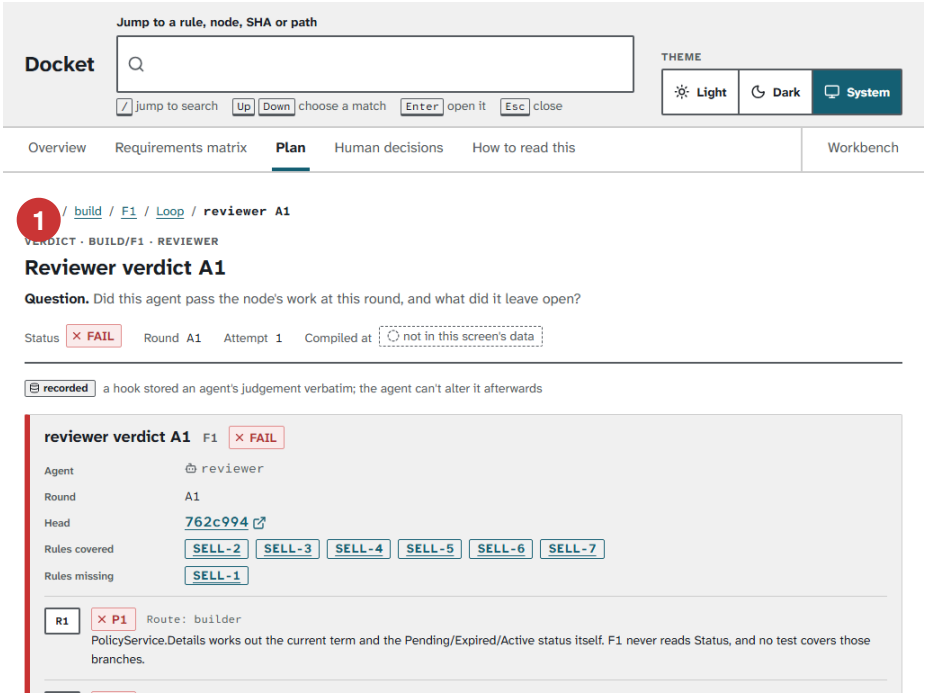
`src/Uinsure.Policies.Web/app/explorer/activity/Activity.tsx:123`

```
export function Activity() {
  const { trace } = useShell();
  const { planId, nodeId } = useParams();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

A verdict

What did the reviewer decide, and why?

One verdict, FAIL here, with the findings behind it. `#/plans/build/F1/verdicts/A1/reviewer` Rules and requirements it proves: **LOOP-3** **DATA-2**



A verdict (light theme, 1024 px wide, `#/plans/build/F1/verdicts/A1/reviewer`)

#	Region (its data-region tag)	Proves
1	Verdict summary	**LOOP-3**

Below this view: Findings; Raw json.

1 Open a verdict from the loop.

You should see: The verdict summary (marker 1).

2 Read its findings.

You should see: Each finding with its rule and file.

3 Open the findings round.

You should see: The findings screen opens.

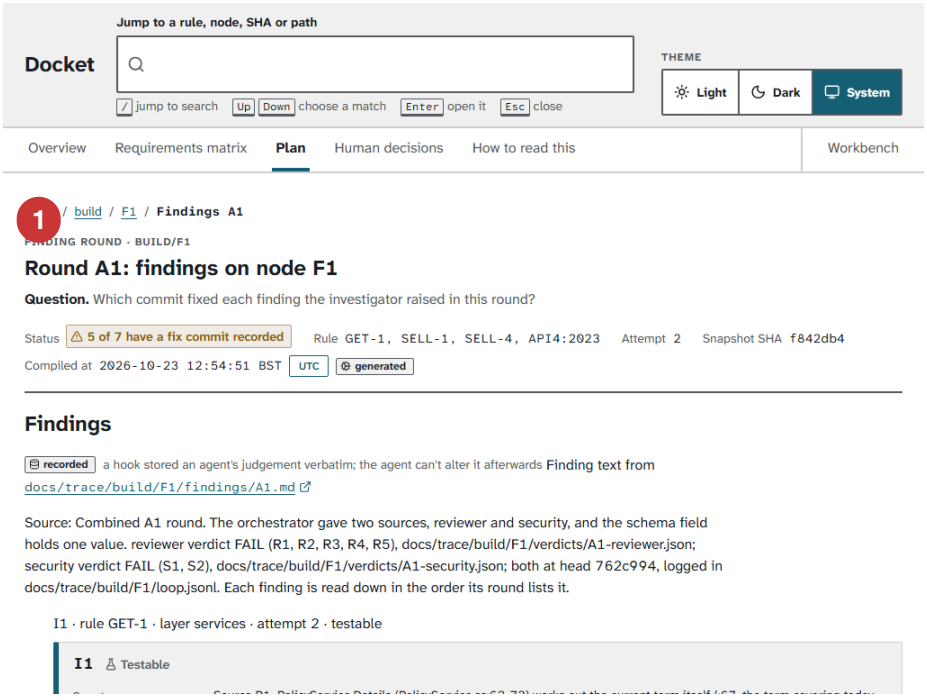
`src/Uinsure.Policies.Web/app/explorer/verdict/Verdict.tsx:182`

```
export function Verdict() {
  const { planId = '', nodeId = '', round = '', agent = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

A findings round

What was wrong, and how was it fixed?

A findings round: each finding, its cause and the fixing commit. `#/plans/build/F1/findings/A1` Rules and requirements it proves: LOOP-4 LOOP-2



A findings round (light theme, 1024 px wide, `#/plans/build/F1/findings/A1`)

#	Region (its data-region tag)	Proves
1	Findings	LOOP-4

Below this view: Fixed by.

1. Open the findings of a round.

   You should see: The findings list (marker 1).

2. Read one finding.

   You should see: Its source and the layer it was in.

3. Follow the fix commit.

   You should see: The commit carries a Fixes-Finding trailer.

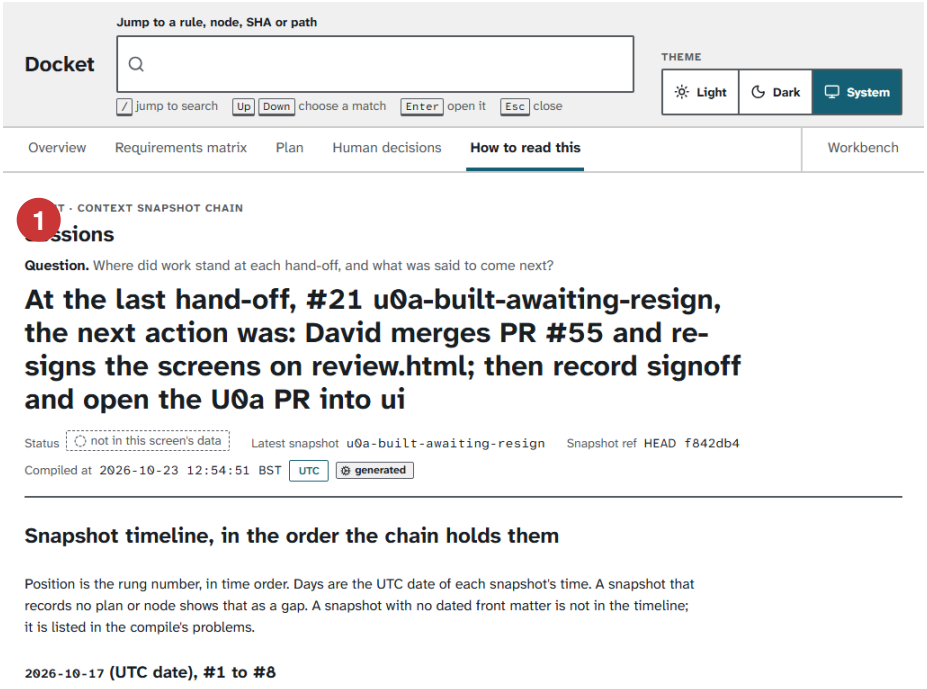
`src/Uinsure.Policies.Web/app/explorer/finding/Finding.tsx:124`

```
export function Finding() {
  const { planId = '', nodeId = '', round = '' } = useParams();
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

Sessions

What did each working session decide?

The timeline of context snapshots. `#/sessions` Rules and requirements it proves: **CTX-1** **NAV-2**



Sessions (light theme, 1024 px wide, `#/sessions`)

#	Region (its data-region tag)	Proves
1	Snapshot timeline	**CTX-1**

Below this view: One snapshot open at its own url sessions file.

1. Open `#/sessions`.

   You should see: Snapshot timeline (marker 1).

2. Pick a snapshot.

   You should see: It opens at its own URL.

3. Copy the URL.

   You should see: It is a deep link.

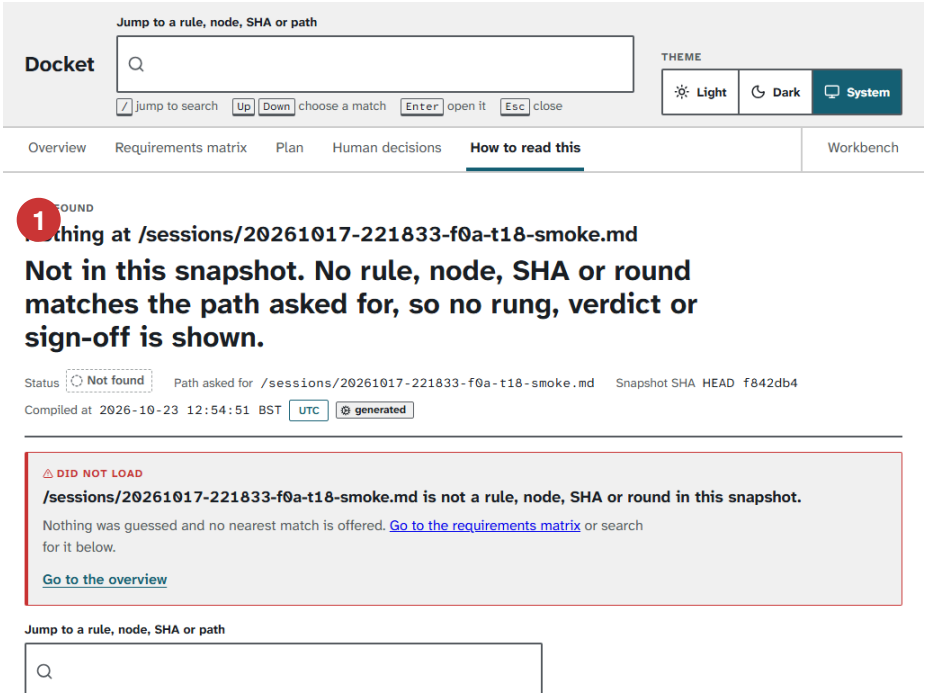
`src/Uinsure.Policies.Web/app/explorer/sessions/Sessions.tsx:52`

```
export function Sessions() {
  const { trace } = useShell();
  const { file } = useParams();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

One session snapshot

What was asked, and where did it stop?

One snapshot open at its own URL. `#/sessions/<file>` Rules and requirements it proves: NAV-2



One session snapshot (light theme, 1024 px wide, `#/sessions/20261017-221833-f0a-t18-smoke.md`)

#	Region (its data-region tag)	Proves
1	What was asked for and where to go	NAV-2

1. Open a snapshot from the timeline.

   You should see: What was asked for and where it stopped (marker 1).

2. Read the next action.

   You should see: The hand-off for the next session.

3. Return to the timeline.

   You should see: The browser's back button works.

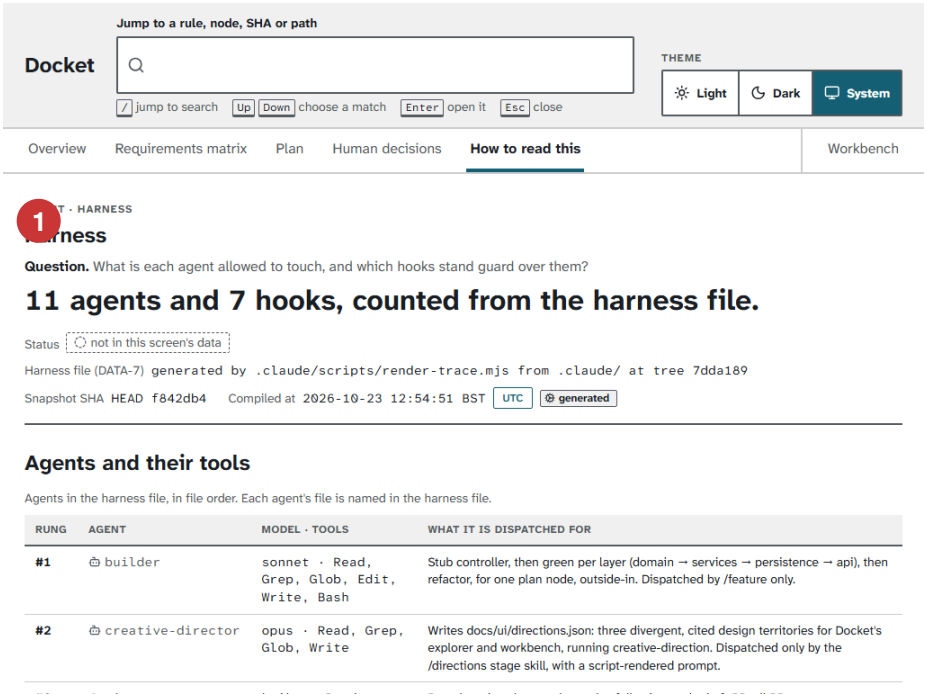
`src/Uinsure.Policies.Web/app/explorer/sessions/Sessions.tsx:121`

```
    <section className="s-u5-region" data-region="one-snapshot-open-at-its-own-url-sessions-file" data-req="NAV-2">
      <h2 className="s-u5-head">{`One snapshot, open: #${row.position} ${row.slug ?? row.file}`}</h2>
      <DataTable
        caption={`Header fields of snapshot #${row.position}, as its front matter records them.`}
```

The harness

Which agents, hooks and rules governed the build?

The agents and hooks, as compiled from the harness files. `#/harness` Rules and requirements it proves: **AGENT-3**



The harness (light theme, 1024 px wide, `#/harness`)

#	Region (its data-region tag)	Proves
1	Agents	AGENT-3

Below this view: Hooks.

1. Open `#/harness`.

   You should see: Agents (marker 1).

2. Scroll to hooks.

   You should see: Each hook and what it guards.

3. Cross-check with a commit's hook agent.

   You should see: The names match.

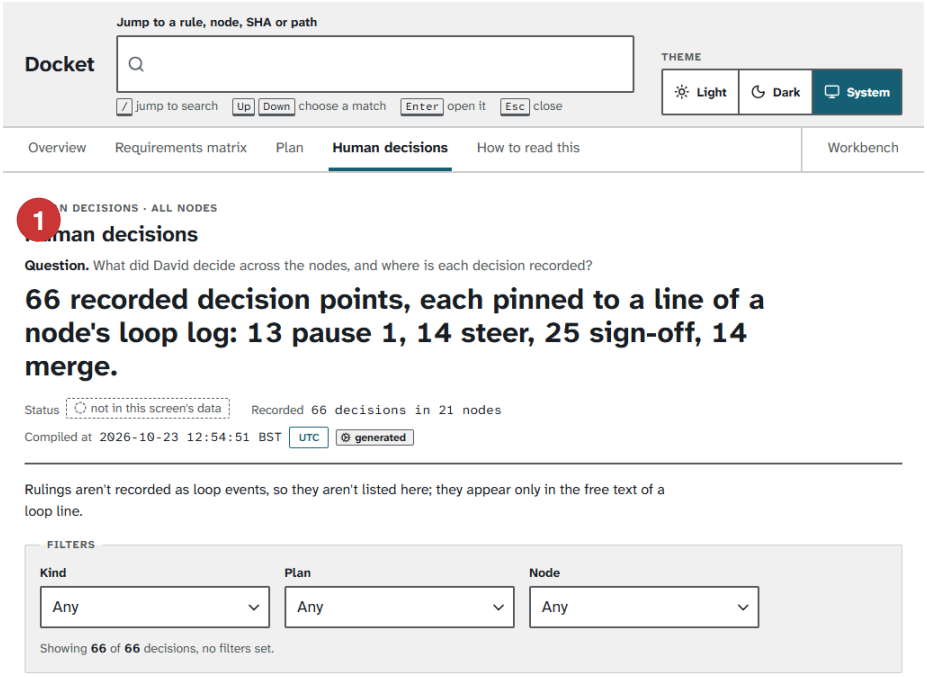
src/Uinsure.Policies.Web/app/explorer/harness/Harness.tsx:30

```
export function Harness() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  const { harness } = trace.trace;
```

Human decisions

Where did a human decide?

Every human decision (pauses, rulings) in time order, with its evidence line. `#/decisions` Rules and requirements it proves: HITL-1



Human decisions (light theme, 1024 px wide, `#/decisions`)

#	Region (its data-region tag)	Proves
1	Decision list	HITL-1

1. Open `#/decisions`.

   You should see: The decision list (marker 1).

2. Read a row.

   You should see: Kind, node, time and the file and line it came from.

3. Follow its evidence link.

   You should see: The source opens.

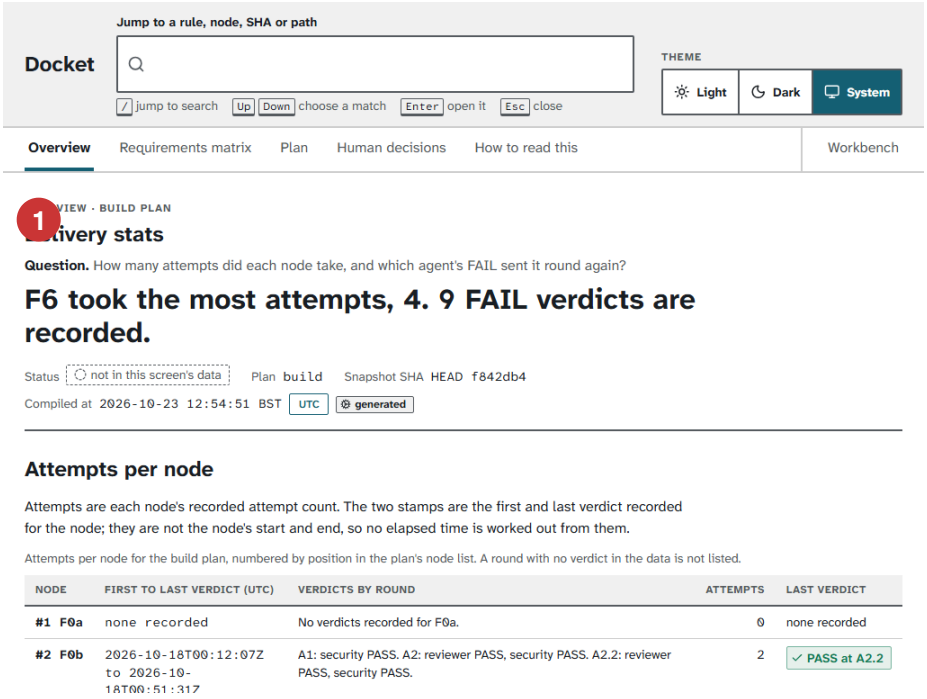
`src/Uinsure.Policies.Web/app/explorer/decisions/Decisions.tsx:42`

```
export function Decisions() {
  const { trace } = useShell();
  const [filters, setFilters] = useState<Filters>(NO_FILTERS);
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
```

Stats

How much did each agent do?

Facts per agent and node, never summed across rules. `#/stats` Rules and requirements it proves: STATS-1



Stats (light theme, 1024 px wide, `#/stats`)

#	Region (its data-region tag)	Proves
1	Facts per agent and node	STATS-1

1. Open `#/stats`.

   You should see: Facts per agent and node (marker 1).

2. Sort or scan a column.

   You should see: Counts come from the trail.

3. Return to the Overview.

   You should see: The figures there use the same source.

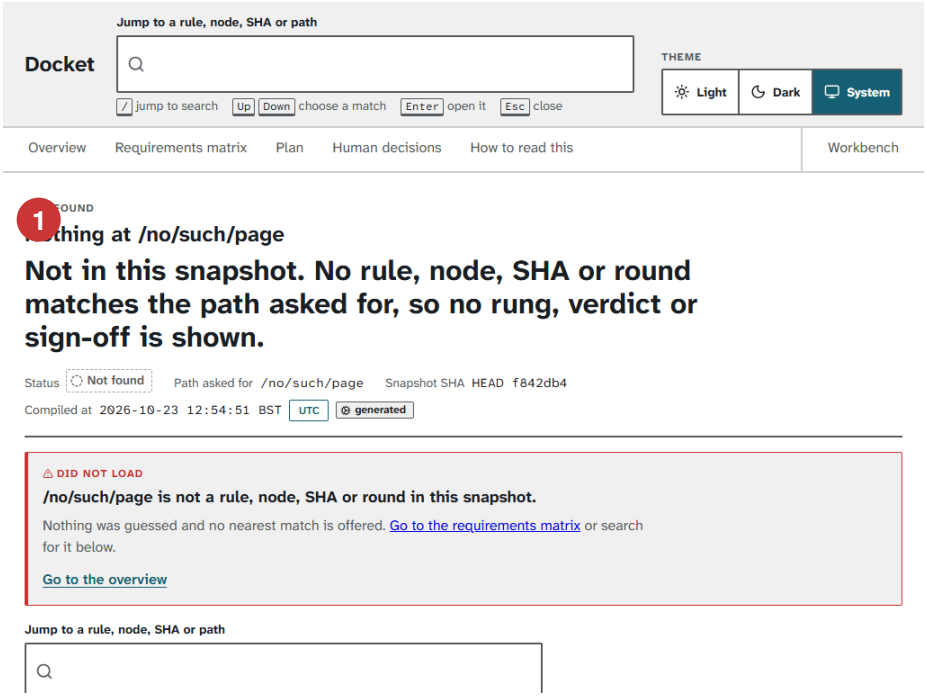
`src/Uinsure.Policies.Web/app/explorer/stats/Stats.tsx:161`

```
export function Stats() {
  const { trace } = useShell();
  if (trace.state !== 'ready') return <DataNotice trace={trace} />;
  return <StatsTables trace={trace.trace} snapshot={trace.snapshot} />;
```

Not found

What if the address is wrong?

An honest empty page: what was asked for and where to go. `#/no/such/page` Rules and requirements it proves: NAV-2



Not found (light theme, 1024 px wide, `#/no/such/page`)

#	Region (its data-region tag)	Proves
1	What was asked for and where to go	NAV-2

1. Open any address that does not exist, for example `#/no/such/page`.

   You should see: A not-found page naming what was asked for (marker 1).

2. Use the nav.

   You should see: Any other screen opens.

3. Fix the address.

   You should see: Deep links are case-sensitive for ids.

`src/Uinsure.Policies.Web/app/shell/NotFound.tsx:9`

```
export function NotFoundPath() {
  return <NotFound asked={useLocation().pathname} />;
}
```

Check your understanding

Question	Answer
What does a marker number refer to?	The region in the key beneath the figure, read from its `data-region` tag.

Question	Answer
Where do I start to check a rule?	The Requirements matrix, then the rule page.

4. Workbench tour

Seven screens that drive the API: sell, view, quote and cancel, claim, renew and inspect.

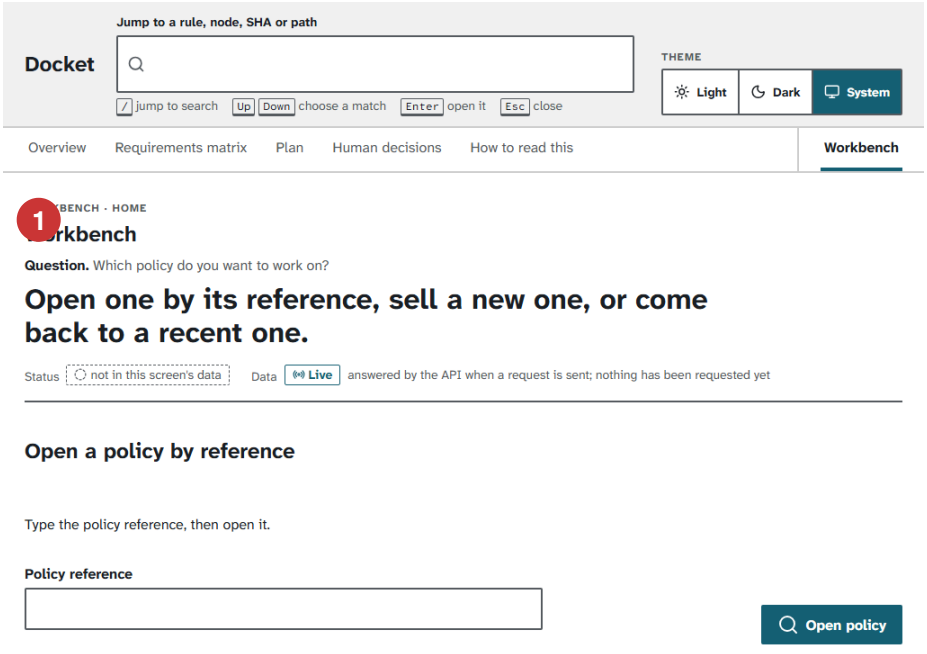
The workbench shows the status the API answered, never a status it did not receive. Each form can send what a user types, and an optional switch lets a request go to the API even when the form's own checks would stop it.

▼ In depth

Workbench home

Which policy do I want to work on?

Open a policy by reference, start a sale, or pick a recent one kept in this browser. `#/workbench` Rules and requirements it proves: `WB-2` `BUSY-1` `WB-1` `WB-9`



Workbench home (light theme, 1024 px wide, `#/workbench`)

#	Region (its data-region tag)	Proves
1	Open a policy by reference	`WB-2`

Below this view: Open by reference busy while the policy loads; Sell a new policy entry; Recent policies kept in this browser.

1. Open `#/workbench`.

   You should see: The open-by-reference field (marker 1); nothing has been sent, so no request exists.

2. Type a reference and press Open policy.

   You should see: One GET `/policies/<reference>` is sent.

3. Or choose Sell a policy.

   You should see: The sell form opens.

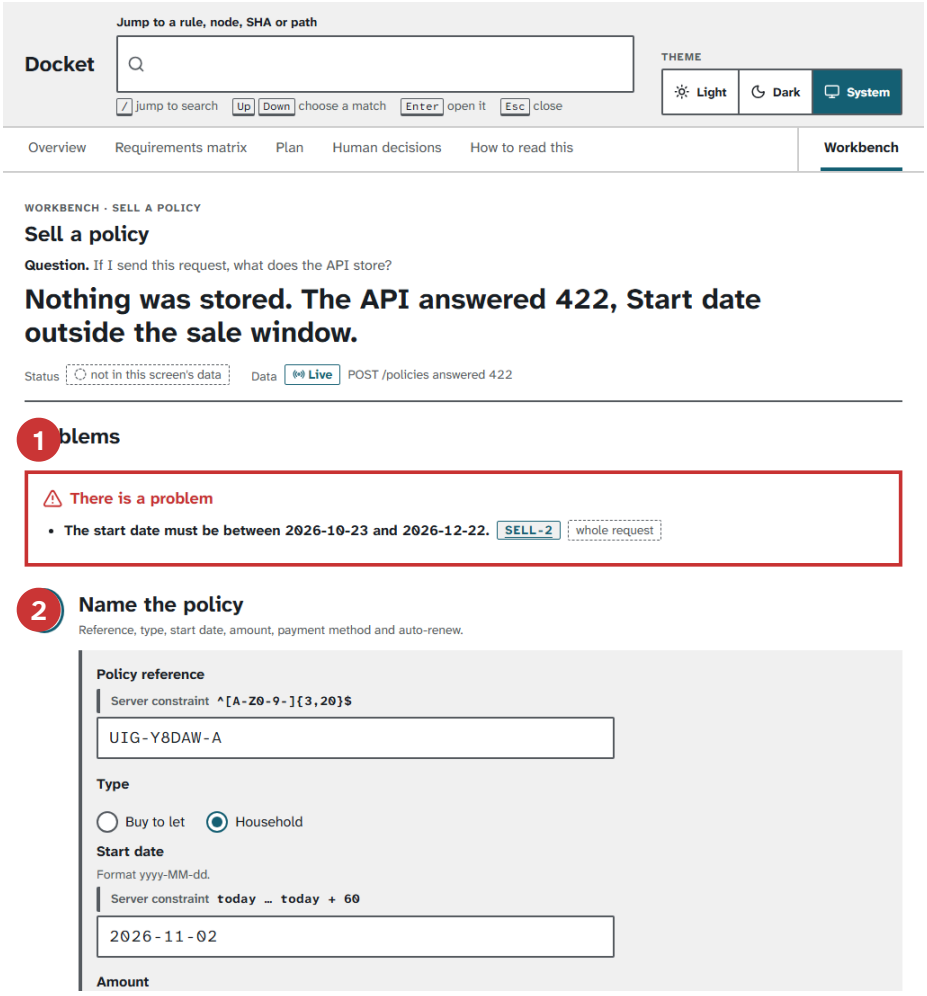
`src/Uinsure.Policies.Web/app/workbench/home/WorkbenchHome.tsx:32`

```
export function WorkbenchHome() {
  const navigate = useNavigate();
  const [reference, setReference] = useState('');
  const [opening, setOpening] = useState<Opening>({ kind: 'idle' });
```

Sell a policy

If I send this request, what does the API store?

The sell request as a form: policy fields, one to three policyholders, the property, and a send-anyway switch. `#/workbench/sell` Rules and requirements it proves: **WB-3** **WB-1** **WB-6** **BUSY-1**



Sell a policy (light theme, 1024 px wide, `#/workbench/sell`)

#	Region (its data-region tag)	Proves
1	Error summary every problem each linked to its field and its rule	WB-3
2	Policy fields reference type start date amount payment method auto renew	WB-1

Below this view: Reference as stored normalisation preview; Policyholders 1 to 3; Property; Send it anyway client checks off; Submit busy while the sell request runs spinner disabled; The request as sent inspector.

1. Open `#/workbench/sell` and fill the policy fields.

   You should see: The reference preview shows it as the API will store it (trimmed, upper case).

2. Fill each policyholder and the property.

   You should see: Add another policyholder up to three.

3. Press Sell this policy.

   You should see: The API answers 201 and the page says so, with a link to the policy.

4 Switch Send anyway on to test a rule.

You should see: The form's own checks are off and the API's answer is shown.

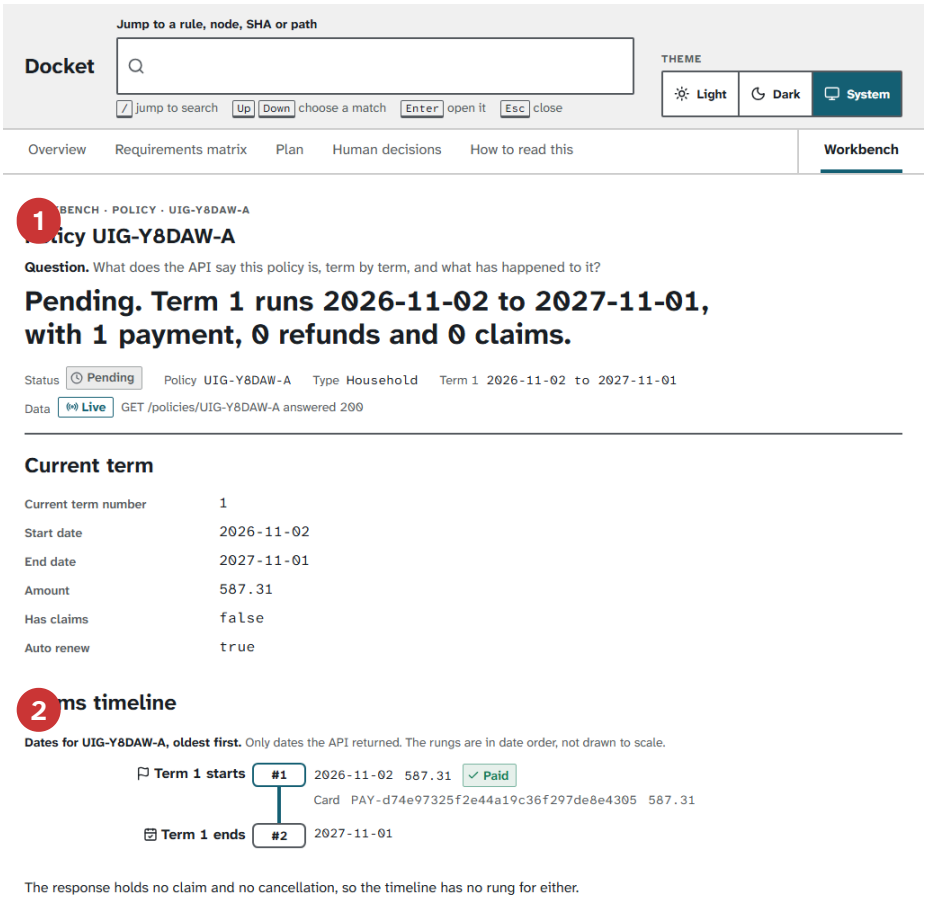
`src/Uinsure.Policies.Web/app/workbench/sell/SellPolicy.tsx:129`

```
export function SellPolicy() {
  const [values, setValues] = useState<Values>(INITIAL);
  const [holders, setHolders] = useState<Holder[]>([blankHolder('holder-1')]);
  const [sendAnyway, setSendAnyway] = useState(false);
```

Policy view

What does the API hold for this policy?

Status and current term, the term timeline, holders and property, the actions, and the requests that touched it. `#/workbench/policies/UIG-Y8DAW-A` Rules and requirements it proves: WB-2 WB-5 WB-4 WB-3 WB-7



Policy view (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-A`)

#	Region (its data-region tag)	Proves
1	Status and current term	WB-2
2	Terms timeline terms payments claims cancellation dates the api returned only	WB-2

Below this view: Policyholders and property; Payments and refunds; Claims; Cancellation summary from get per term breakdown from the logged cancel response; Record level problem banner 404 409; Actions quote and cancel record a claim renew; Requests that touched this policy.

1. Open the policy from the sell answer or the home.

   You should see: Status and current term (marker 1).

2. Read the timeline and payments.

   You should see: Each term with its payment.

3. Use an action button.

   You should see: Quote and cancel, Record a claim, Renew.

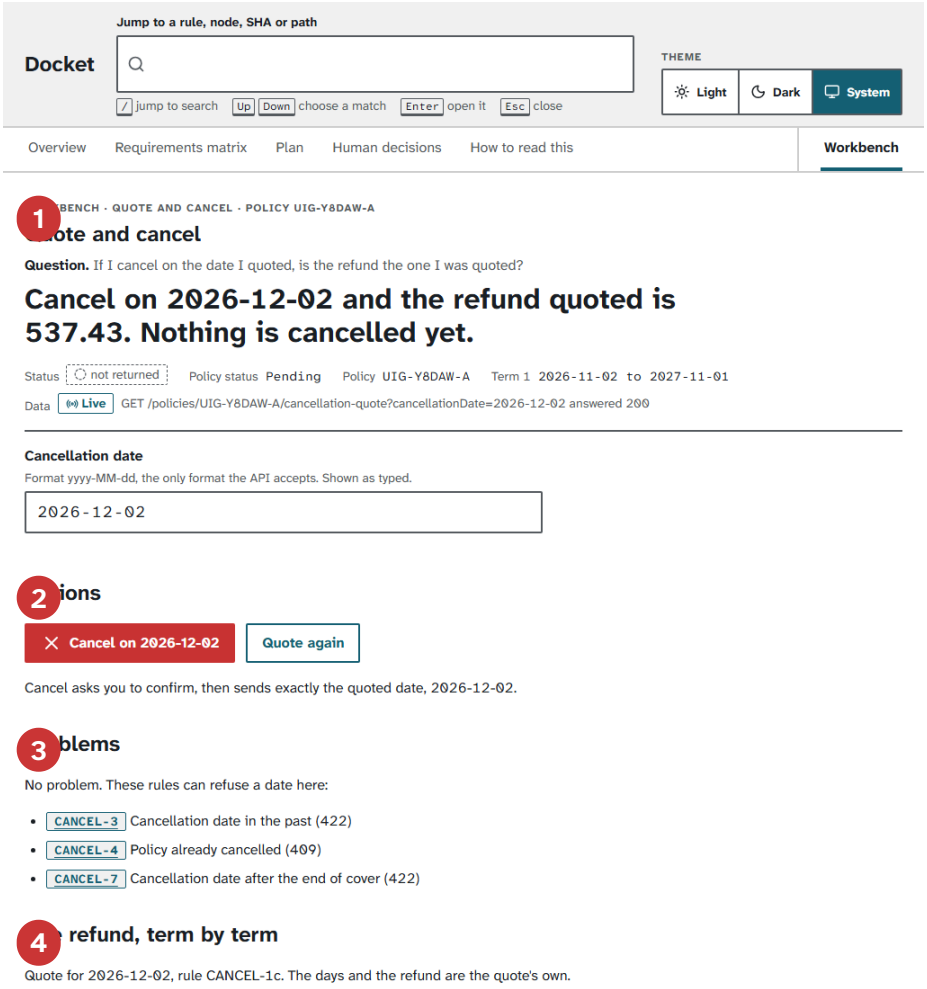
`src/Uinsure.Policies.Web/app/workbench/policy/PolicyView.tsx:325`

```
export function PolicyView() {
  const { reference = '' } = useParams();
  const cancelPath = cancelRoute(reference);
  const navigate = useNavigate();
```

Quote and cancel

If I cancel on the date I quoted, is the refund the one I was quoted?

Quote a date, read the refund term by term, then cancel on that date after a confirmation. `#/workbench/policies/UIG-Y8DAW-A/cancel` Rules and requirements it proves: **WB-4** **BUSY-1** **WB-3**



Quote and cancel (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-A/cancel`)

#	Region (its data-region tag)	Proves
1	Cancellation date and quote	WB-4
2	Quote and cancel actions each busy while its request runs	BUSY-1
3	Problems cancel 3 cancel 4 cancel 7	WB-3
4	The refund as a worked sum per term with the rule applied	WB-4

Below this view: Cancel with the quoted date then quote and cancellation compared; The requests as sent inspector.

1. Open Quote and cancel and type a cancellation date.

   You should see: Format yyyy-MM-dd, shown as typed.

2. Press Quote this date.

   You should see: The refund as a worked sum per term, with the rule applied (CANCEL-1a, 1b or 1c).

3. Press Cancel on that date and confirm.

   You should see: A confirmation dialog, then the API's 201 and a re-read policy status.

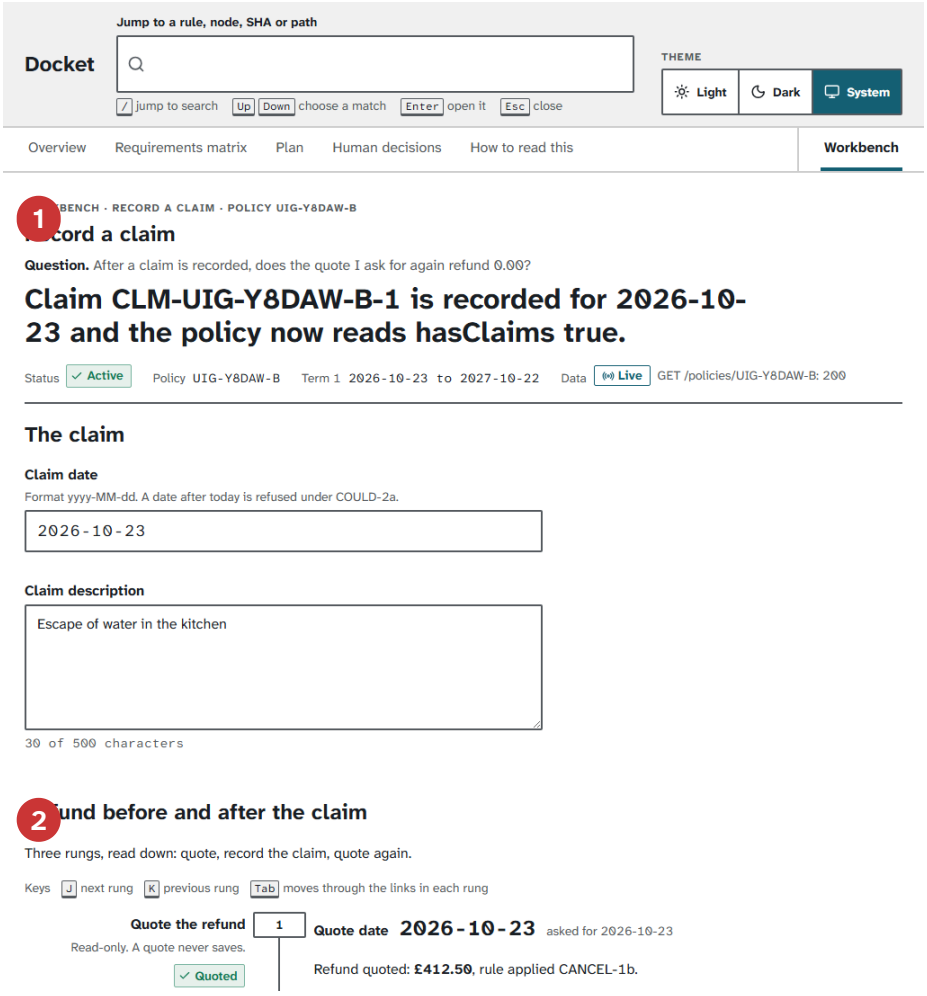
`src/Uinsure.Policies.Web/app/workbench/cancel/CancelPolicy.tsx:64`

```
export function CancelPolicy() {
  const { reference = '' } = useParams();
  const [policy, setPolicy] = useState<Policy | null>(null);
  const [date, setDate] = useState('');
```

Record a claim

What does a claim do to my refund?

Quote, record the claim, quote again: the refund before and after. `#/workbench/policies/UIG-Y8DAW-B/claims/new` Rules and requirements it proves: **WB-5** **WB-3** **BUSY-1**



Record a claim (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-B/claims/new`)

#	Region (its data-region tag)	Proves
1	Claim date and description	WB-5

#	Region (its data-region tag)	Proves
2	Refund before and after the claim two quotes	WB-5

Below this view: Problems could 2a could 2b could 2c; Record claim busy while the request and the two quotes run.

1. Open Record a claim and fill the date and description.

   You should see: A date after today is refused (COULD-2a).

2. Press Record this claim.

   You should see: The claim is recorded (201).

3. Read the two quotes.

   You should see: Any claim makes the later refund 0.00 (COULD-2).

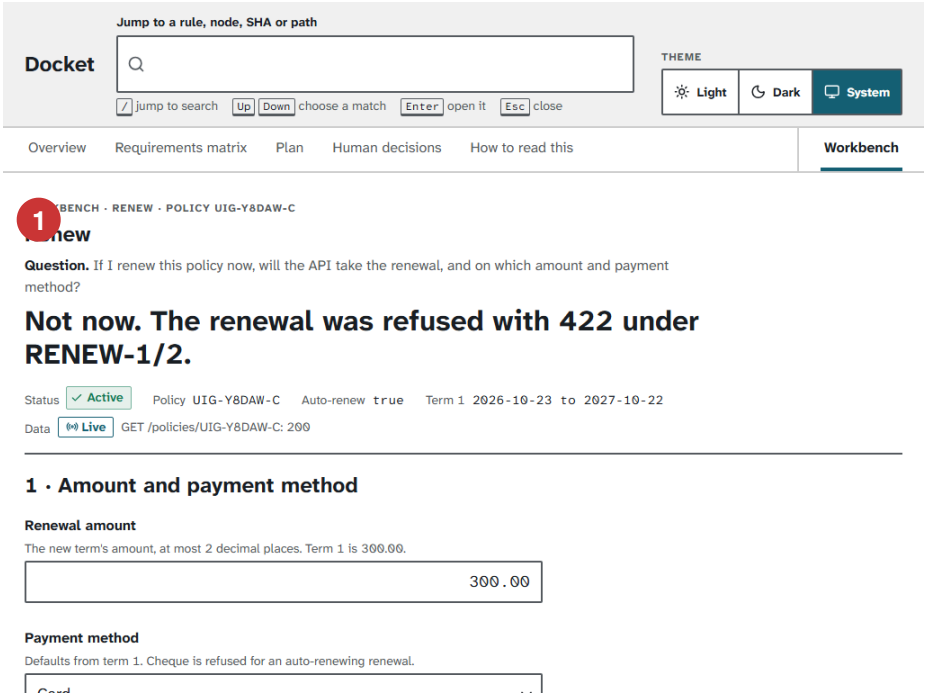
`src/Uinsure.Policies.Web/app/workbench/claim/RecordClaim.tsx:84`

```
export function RecordClaim() {
  const { reference = '' } = useParams();
  const [policy, setPolicy] = useState<ApiAnswer<Policy> | null>(null);
  const [attempt, setAttempt] = useState(0);
```

Renew

Will the API take the renewal?

Renewal amount and payment method, and the problems the API may answer. `#/workbench/policies/UIG-Y8DAW-C/renew` Rules and requirements it proves: WB-5 WB-3 WB-2 BUSY-1



Renew (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-C/renew`)

#	Region (its data-region tag)	Proves
1	Renewal amount and payment method default shown from the current term omitted when auto …	WB-5

Below this view: Problems renew 1 2 with its window renew 3b renew 5 renew 6 could 3 several at once list…; The new term last on the timeline; Renew busy while the request runs.

1. Open Renew for a policy.

   You should see: Amount and payment method default from the current term.

2. Press Renew policy.

   You should see: On a new policy the API answers 422: renewal opens 30 days before the end date (RENEW-1, RENEW-2).

3. Read the problem list.

   You should see: Every violation is listed, none hidden.

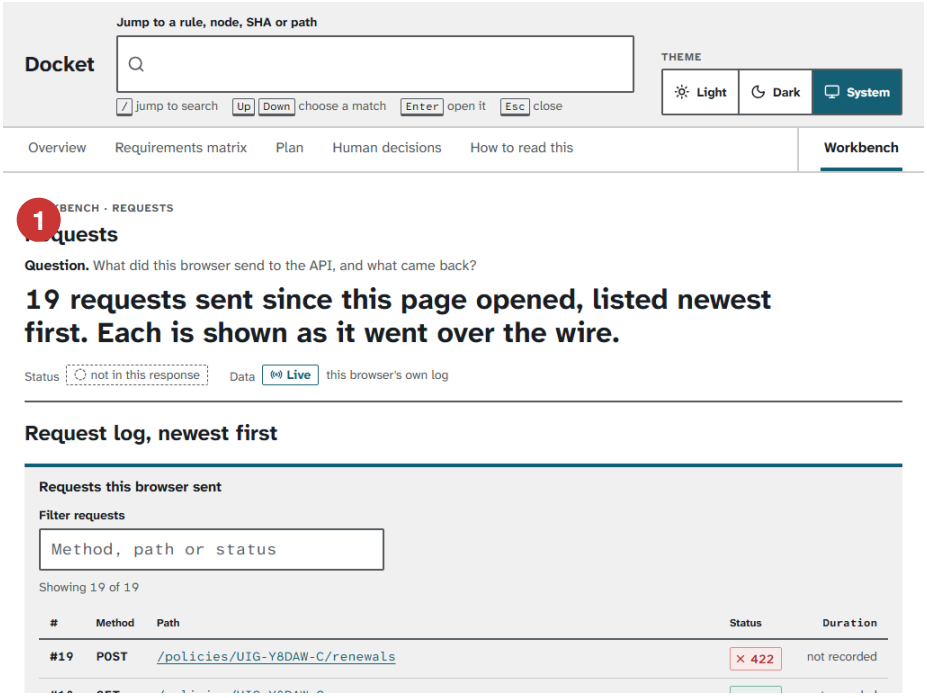
`src/Uinsure.Policies.Web/app/workbench/renew/RenewPolicy.tsx:61`

```
export function RenewPolicy() {
  const { reference = '' } = useParams();
  const [policy, setPolicy] = useState<ApiAnswer<Policy> | null>(null);
  const [attempt, setAttempt] = useState(0);
```

Requests

What did this browser send, and what came back?

The request log, newest first, and one exchange as it went over the wire, with a curl.
`#/workbench/requests` Rules and requirements it proves: **WB-7**



Requests (light theme, 1024 px wide, `#/workbench/requests`)

#	Region (its data-region tag)	Proves
1	Request log	WB-7

Below this view: One exchange request response status line curl.

1. Open `#/workbench/requests`.

   You should see: The request log (marker 1).

2. Select a row.

   You should see: One exchange: request, response, status line and a curl.

3. Copy the curl.

   You should see: It repeats the request against this origin.

`src/Uinsure.Policies.Web/app/workbench/requests/RequestLogPage.tsx:23`

```
export function RequestLogPage() {
  const log = useRequestLog();
  const [picked, setPicked] = useState<number | null>(null);
```

Check your understanding

Question	Answer
Which screen sends a request to sell a policy?	Sell a policy, `#/workbench/sell`.
What is Send anyway for?	Skip the form's own checks so the API's rule answer can be seen.

5. Workflows, end to end

Eight tasks, each from first click to last, run against the live API.

The screenshots in this section were taken during one run with references `UIG-Y8DAW-A`, `UIG-Y8DAW-B` and `UIG-Y8DAW-C`. Statuses seen: sell 201, duplicate 409, quote 200, cancel 201, claim 201, renew 422.

▼ In depth

Sell a policy, then view it

1. Open the sell form and fill it in.

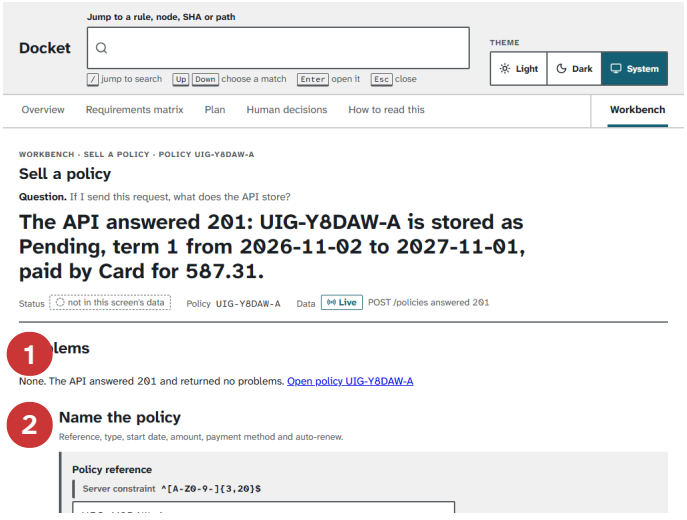
   You should see: The form as filled, with two policyholders.

2. Press Sell this policy.

   You should see: The API answers 201; the page links the policy.

3. Follow Open policy.

   You should see: The policy view with status and term (see the Policy view above).



The sell form after the API accepted the policy (light theme, 1024 px wide, `#/workbench/sell`)

#	Region (its data-region tag)	Proves
1	Error summary every problem each linked to its field and its rule	WB-3
2	Policy fields reference type start date amount payment method auto renew	WB-1

Below this view: Reference as stored normalisation preview; Policyholders 1 to 3; Property; Send it anyway client checks off; Submit busy while the sell request runs spinner disabled; The request as sent inspector.

Cancel with a quote

1. From the policy view press Quote and cancel, then type a date and press Quote this date.

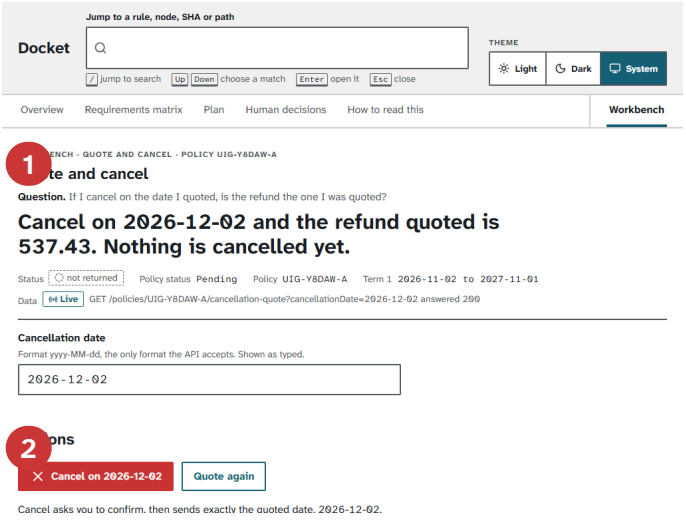
   You should see: The quote answers 200 for 2026-12-02.

2. Press Cancel on that date.

   You should see: A confirmation dialog first.

3. Confirm.

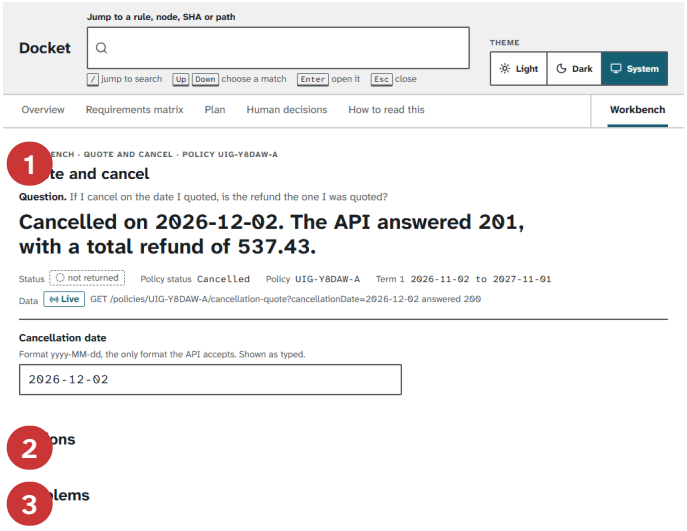
   You should see: The API answers 201 and the policy status is re-read as Cancelled.



The confirmation dialog (no data-region tag; see the spec's audit) (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-A/cancel`)

#	Region (its data-region tag)	Proves
1	Cancellation date and quote	WB-4
2	Quote and cancel actions each busy while its request runs	BUSY-1

Below this view: Problems cancel 3 cancel 4 cancel 7; The refund as a worked sum per term with the rule applied; Cancel with the quoted date then quote and cancellation compared; The requests as sent inspector.



The cancel screen after the cancellation (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-A/cancel`)

#	Region (its data-region tag)	Proves
1	Cancellation date and quote	WB-4
2	Quote and cancel actions each busy while its request runs	BUSY-1
3	Problems cancel 3 cancel 4 cancel 7	WB-3

Below this view: The refund as a worked sum per term with the rule applied; Cancel with the quoted date then quote and cancellation compared; The requests as sent inspector.

Make a claim

1. Open Record a claim from a policy that has started.

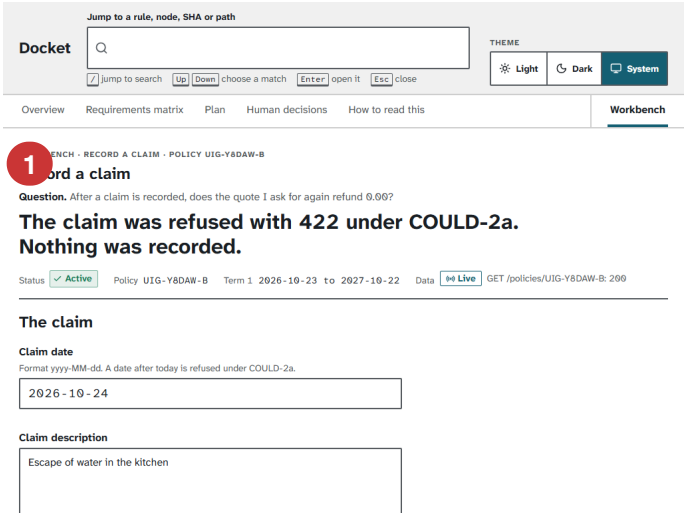
   You should see: The claim form.

2. Enter a date after today and press Record this claim.

   You should see: The API answers 422 (COULD-2a).

3. Enter today's date and record again.

   You should see: The API answers 201; the refund after the claim is shown beside the refund before it.



A claim dated tomorrow, refused (light theme, 1024 px wide, `#/workbench/policies/UIG-Y8DAW-B/claims/new`)

#	Region (its data-region tag)	Proves
1	Claim date and description	WB-5

Below this view: Refund before and after the claim two quotes; Problems could 2a could 2b could 2c; Record claim busy while the request and the two quotes run.

Renew

1. Open Renew on a policy and press Renew policy.

   You should see: A new policy answers 422: renewal opens 30 days before the end date.

2. Read the problems list.

   You should see: RENEW-1 and RENEW-2 name the window.

3. Renew a policy inside its window.

   You should see: Not shown: on the real clock a policy sold today is a year from its window.

Read a rule's evidence from the requirement to its tests and commits

1. Type `CANCEL-1b` in the jump-to-search box and press Enter.

   You should see: The rule page: the requirement and its agreed interpretation.

2. Read the red and green evidence.

   You should see: The red commit precedes the green commit.

3. Open a commit.

   You should see: Its trailers, its TDD step and its diff.

Follow a node's walkthrough

1. Open Plan, then a node.

   You should see: Completeness and the rules it covers.

2. Open its Walkthrough tab.

   You should see: Commits in order, tagged red, stub, green.

3. Open the Loop tab.

   You should see: The attempt ladder.

Read a verdict and its findings

1. On the Loop tab choose a FAIL verdict.

   You should see: The verdict page.

2. Open its findings round.

   You should see: Each finding with its cause.

3. Follow the fix commit.

   You should see: It carries a Fixes-Finding trailer.

Inspect a request

1. Press any workbench action, then open `#/workbench/requests`.

   You should see: The log lists the request.

2. Select it.

   You should see: The exchange as it went, with its status line.

3. Copy its curl and run it.

   You should see: The same answer comes back.

Check your understanding

Question	Answer
Which statuses did the sell, duplicate and renew attempts get?	201, 409 and 422.
Why can't a new policy be renewed?	RENEW-1 and RENEW-2: the window opens 30 days before the end date.

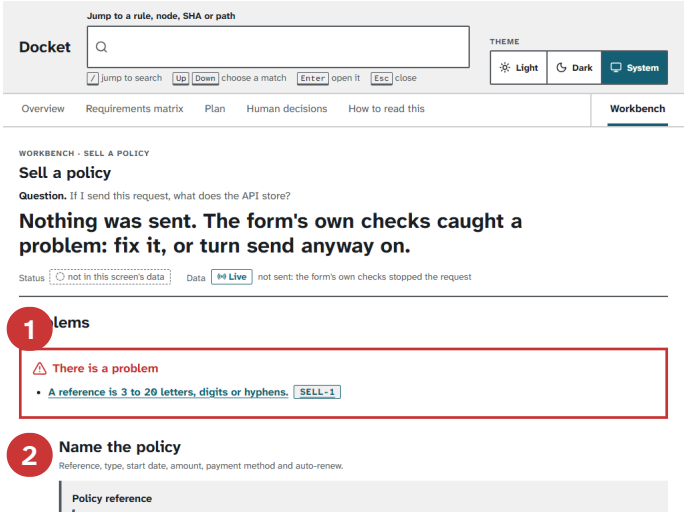
6. Error and edge states

Every failure is shown as the API or the browser reported it.

A 4xx or 422 is a Problem banner with the status, rule id and each violation. When the API cannot be reached the page says no answer was read. Empty states say so and send no request.

▼ In depth

State	What you see	Shot
Form check (client)	The form stops a bad reference before anything is sent	below
422, SELL-2	"Nothing was stored. The API answered 422"	below
409, SELL-1	A duplicate reference is refused	below
404, GET-1	Opening a policy that does not exist	below
API unreachable	No response was read	below
Empty	No request sent; no recent policies	below

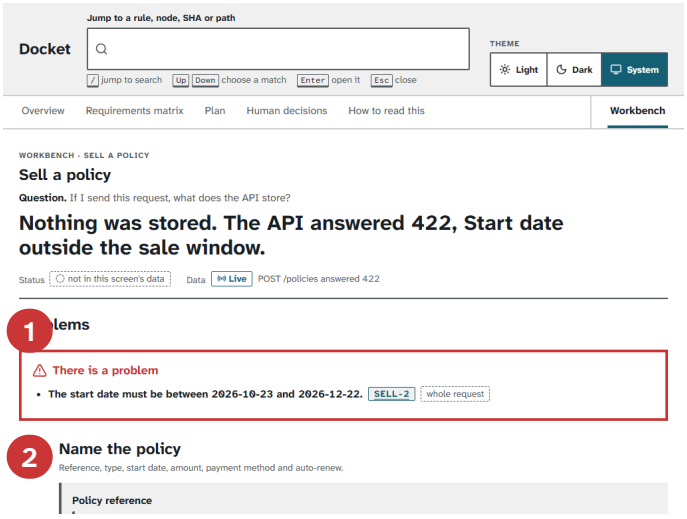


The form's own check (light theme, 1024 px wide, `#/workbench/sell`)

#	Region (its data-region tag)	Proves
1	Error summary every problem each linked to its field and its rule	WB-3

#	Region (its data-region tag)	Proves
2	Policy fields reference type start date amount payment method auto renew	WB-1

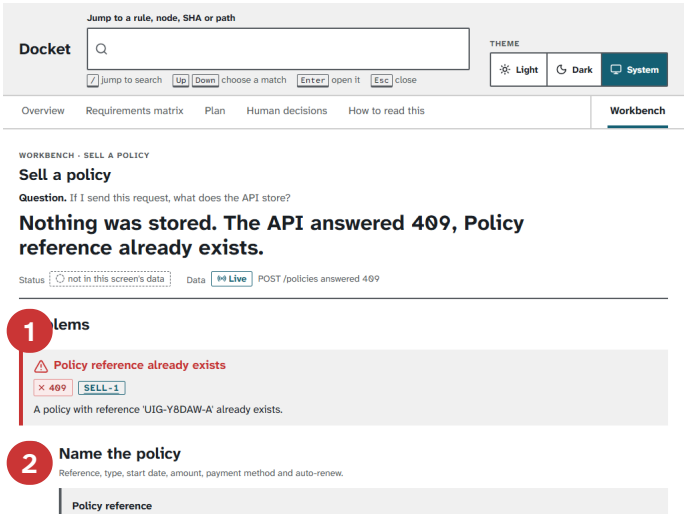
Below this view: Reference as stored normalisation preview; Policyholders 1 to 3; Property; Send it anyway client checks off; Submit busy while the sell request runs spinner disabled; The request as sent inspector.



422: a backdated start (light theme, 1024 px wide, `#/workbench/sell`)

#	Region (its data-region tag)	Proves
1	Error summary every problem each linked to its field and its rule	WB-3
2	Policy fields reference type start date amount payment method auto renew	WB-1

Below this view: Reference as stored normalisation preview; Policyholders 1 to 3; Property; Send it anyway client checks off; Submit busy while the sell request runs spinner disabled; The request as sent inspector.

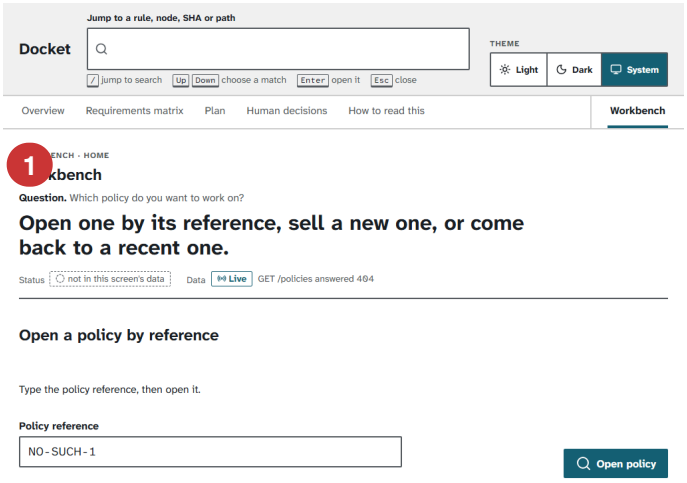


409: the same reference again (light theme, 1024 px wide, `#/workbench/sell`)

#	Region (its data-region tag)	Proves
1	Error summary every problem each linked to its field and its rule	WB-3

#	Region (its data-region tag)	Proves
2	Policy fields reference type start date amount payment method auto renew	WB-1

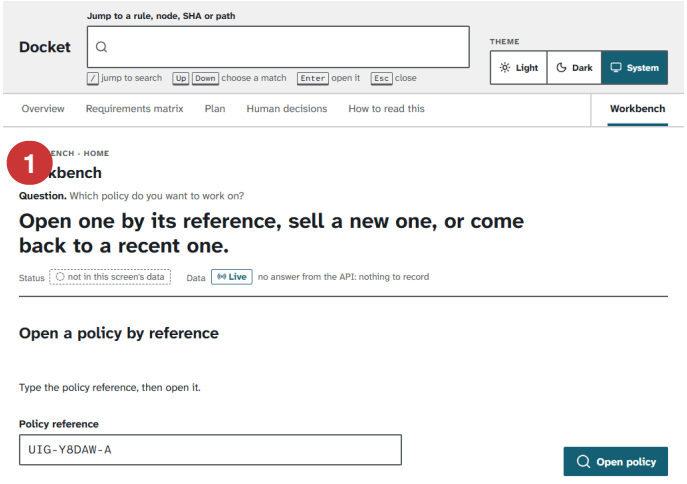
Below this view: Reference as stored normalisation preview; Policyholders 1 to 3; Property; Send it anyway client checks off; Submit busy while the sell request runs spinner disabled; The request as sent inspector.



404: no such policy (light theme, 1024 px wide, `#/workbench`)

#	Region (its data-region tag)	Proves
1	Open a policy by reference	WB-2

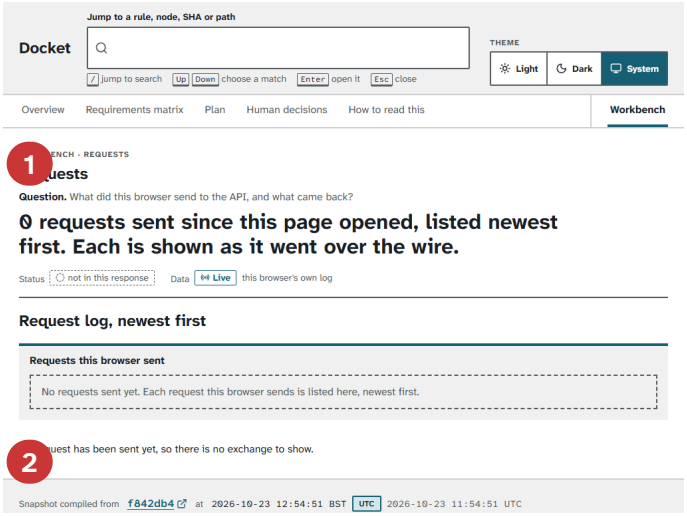
Below this view: Open by reference busy while the policy loads; Sell a new policy entry; Recent policies kept in this browser.



The API unreachable (light theme, 1024 px wide, `#/workbench`)

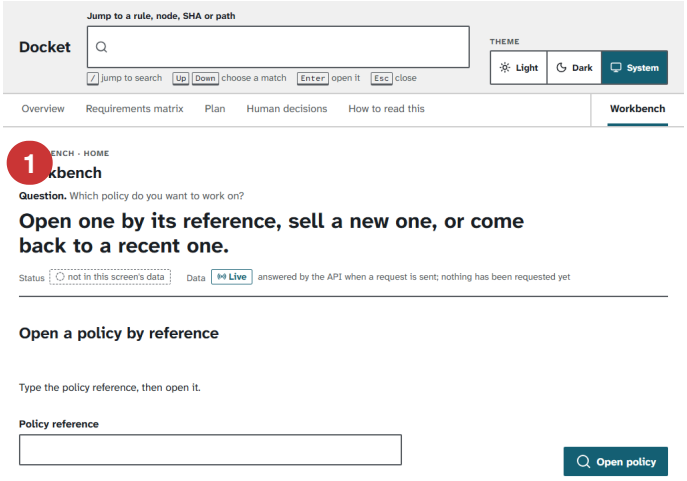
#	Region (its data-region tag)	Proves
1	Open a policy by reference	WB-2

Below this view: Open by reference busy while the policy loads; Sell a new policy entry; Recent policies kept in this browser.



Empty: no request yet (light theme, 1024 px wide, `#/workbench/requests`)

#	Region (its data-region tag)	Proves
1	Request log	WB-7
2	One exchange request response status line curl	WB-7



A recent policy listed (compare with the empty home above) (light theme, 1024 px wide, `#/workbench`)

#	Region (its data-region tag)	Proves
1	Open a policy by reference	WB-2

Below this view: Open by reference busy while the policy loads; Sell a new policy entry; Recent policies kept in this browser.

Check your understanding

Question	Answer
How do I tell a 422 from a 409?	422 is a rule (SELL-2 here); 409 is a conflict (SELL-1 duplicate).

Question	Answer
What does the page show when the API is down?	That no response was read.

7. Accessibility

Keyboard first, two themes, and axe on every screen.

Every control is reachable with the keyboard: press `/` to jump to search, `Up` and `Down` to choose, `Enter` to open and `Esc` to close. The theme toggle sets Light, Dark or System. The Playwright suite runs axe on the screens; the activity and event tables are text equivalents of the lanes.

▼ In depth

The suites are the `a11y-1` specs in tests/Uinsure.Policies.Web.Tests/e2e/U0b/a11y-1.spec.ts:1, and the same name under U8 and W1.

Check your understanding

Question	Answer
Which key opens search?	`/`.
What does axe check?	Contrast, names, roles and landmarks on each screen, in the end-to-end suite.

8. Run the tests yourself

Postman for the API's requests, Playwright for the screens, Vitest for the units.

The workbench sends the same requests the Postman collection sends. Every runner is below with its command; only the Postman run was executed for this guide, and its summary is copied from that run.

▼ In depth

Postman app (MAN-1)

1. Import `tests/postman/Uinsure.Policies.postman_collection.json` and `tests/postman/local.postman_environment.json`.

   You should see: Folders F0b Walking skeleton, F1 Sell, F1b Layered models, F2 Retrieve, F3 Cancel, F4 Renew.

2. Select the local environment, start the API on `5080`, and open the Collection Runner.

   You should see: Run the collection, one folder or one request.

3. Read the results.

   You should see: Each test name starts with its rule id.

Postman CLI (MAN-2)

1. Run one folder against the running API (the base URL is overridden here to the API port in use).

```
postman collection run tests/postman/Uinsure.Policies.postman_collection.json
-e tests/postman/local.postman_environment.json --env-var
baseUrl=http://localhost:5096 -i "F2 Retrieve"
```

   You should see: A summary table.

```
┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│                requests │                 25 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│            test-scripts │                 14 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│      prerequest-scripts │                 19 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│              assertions │                 26 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 1867 ms                                       │
└───────────────────────────────────────────────────────────────────┘
```

Playwright (MAN-5)

1. Install the browser once.

```
npx playwright install chromium
```

   You should see: Chromium downloaded.

```
(not run in this short form)
```

2. Run everything: it builds, starts the app and uses its own API.

```
npm --prefix src/Uinsure.Policies.Web run e2e
```

You should see: Specs pass.

```
(not run in this short form)
```

3. Run one spec or one test by title, rule id or screen name; use `--ui` for UI mode, `--headed` or `--debug` to watch.

```
npx playwright test --config src/Uinsure.Policies.Web/playwright.config.ts
W1/wb-1 -g "WB-1"
```

You should see: The matching tests only.

```
(not run in this short form)
```

4. Open the report and traces.

```
npx playwright show-report
```

You should see: The HTML report; a trace opens in the trace viewer.

```
(not run in this short form)
```

Visual regression
The baselines are Windows renders in `docs/ui/baselines`, so the regression folder is skipped on Linux. Update them only deliberately, with `--update-snapshots`, after a signed-off design change, never to silence a failure. See src/Uinsure.Policies.Web/playwright.config.ts:9.

Vitest (MAN-6)

1. Run the unit tests once, in watch mode, or for one file.

```
npm --prefix src/Uinsure.Policies.Web run test
npx vitest --config src/Uinsure.Policies.Web/vite.config.ts
npx vitest run --config src/Uinsure.Policies.Web/vite.config.ts
tests/Uinsure.Policies.Web.Tests/unit/client/wb-1.test.ts
```

You should see: A pass count.

```
(not run in this short form)
```

Every runner (MAN-7)

Runner	Tests	Use when	Command or click	Needs running	Time
Postman app	The API's requests, by rule	Exploring	Collection Runner	API on 5080	Not measured
Postman CLI	Same	Scripted	`postman collection run …`	API	1867 ms (the F2 Retrieve folder)
Playwright	Screens, a11y, parity, regression	Before a UI change lands	`npm run e2e`	Starts its own	Not measured
Vitest	Units and kit	While coding	`npm run test`	Nothing	Not measured
Visual Studio, `dotnet test`	The .NET suites (see the API guide)	API work	Test Explorer	Nothing	Not measured

Check your understanding

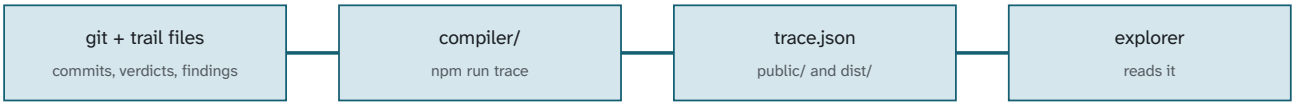
Question	Answer
Which runner needs the API already running?	Postman, app or CLI.
When may I update a visual baseline?	Only deliberately, after a signed-off design change.

9. Understand it

A small folder tree, one compiler, one kit, screens as contracts, five kinds of test.

The explorer's data is compiled once into `trace.json`; the kit supplies components and tokens; the screens were agreed as static HTML before they were built.

▼ In depth



From git to the screen

Topic	Where	What
Folder tree after U10	`src/Uinsure.Policies.Web`: `app/` (explorer, workbench, shell, data), `kit/`, `compiler/`, `public/`; tests in `tests/Uinsure.Policies.Web.Tests`: `unit/`, `kit/`, `e2e/`, `fixtures/`	One project, tests beside the API's
The compiler	`compiler/index.ts`, `git.ts`, `plans.ts`, `privacy.ts`	Reads git and the trail, scrubs private data, writes `trace.json`
The kit and tokens	`kit/`, `tokens.css`	Components (patterns, layouts) and the colours and type; this guide uses the same tokens
Design pipeline	`docs/ui/`: directions, concept, styles, mockups, screens	Directions, then a style, then one static HTML per screen state
How screens are tested	`tests/Uinsure.Policies.Web.Tests`	Unit and kit (Vitest), end to end (Playwright), axe, parity against the screens, visual regression

> **Why**
> Screens are contracts, not code. Choice: agree each screen as static HTML in `docs/ui/screens` and test the app for parity. Alternative: design in the code. Reason: a reviewer can approve a screen without reading React, and a test can fail when the app drifts from it.

`src/Uinsure.Policies.Web/app/shell/Rung.tsx:17`

```
  const headingId = useId();
  return (
    <section className="s-rung" data-region={region} data-req={req} aria-labelledby=
{headingId}>
      <div className="s-label">
        <p className="s-ordinal">{ordinal}</p>
        <h2 className="s-heading" id={headingId}>
```

Check your understanding

Question	Answer
What does the compiler produce?	`trace.json`.
Why static screens first?	So a design is approved before it is coded, and tested for parity afterwards.

10. Troubleshooting

Most problems are a missing API, a missing trace or a missing browser.

Built from the problems met while capturing this guide and from the repo's own configuration.

▼ In depth

Symptom	Cause	Fix
The workbench says no response was read	The API is not running, or `DOCKET_API_PORT` names the wrong port	Start the API; set `DOCKET_API_PORT` to its port and restart the preview
Explorer screens say data is not available	`trace.json` was not compiled	`npm --prefix src/Uinsure.Policies.Web run trace`, then rebuild
Port 4170 in use	The dev server uses `--strictPort`	Stop the other process or pass another `--port`
Playwright says the browser is missing	Chromium not installed	`npx playwright install chromium`
Visual regression fails on Linux or macOS	Baselines are Windows renders; Linux skips the folder	Run on Windows; update baselines only deliberately
Renew answers 422 on a new policy	RENEW-1 and RENEW-2: the window opens 30 days before the end date	Expected; renew inside the window
A sell answers 409	SELL-1: the reference exists (case-insensitive)	Use a new reference
The request log is empty after a reload	It lists requests since the page opened	Send the request again